

武汉大学

WUHAN UNIVERSITY



F7LY-OS 设计文档

参赛队名	F7LY
队伍成员	曹子宸、郑遮宇、官恺祺
指导老师	蔡朝晖

目 录

第一章 概 述	1
1.1 F7LY 内核的介绍	1
1.2 主要工作	1
1.2.1 参考的代码	1
1.2.2 我们的工作	2
1.3 整体架构	3
第二章 内核详细介绍	6
2.1 机器启动	6
2.1.1 RISC-V 启动流程	6
2.1.2 LoongArch 启动流程	8
2.1.3 双架构支持特色	9
2.2 中断管理器	9
2.3 内存管理	11
2.3.1 地址空间	11
2.3.2 物理内存管理	13
2.4 进程管理	20
2.4.1 进程控制块 (PCB)	20
2.4.2 进程调度	22
2.4.3 进程内存管理器	24
2.4.4 进程管理器	26
2.5 文件系统架构	28
2.5.1 虚拟文件系统	28
2.5.2 VFS 核心元数据结构剖析	31
2.5.3 系统文件访问	35
2.6 进程间通信	36
2.6.1 信号机制	36
2.6.2 信号处理函数	37
2.6.3 Futex	39
2.6.4 共享内存机制	40
第三章 系统调用	46

3.1 系统调用概述	46
3.2 系统调用流程	47
3.2.1 系统调用执行流程	47
3.2.2 参数获取机制	47
3.2.3 系统调用表初始化	49
3.2.4 系统调用分发器	50
3.3 系统调用实现	51
3.3.1 进程生命周期管理	51
3.3.2 内存管理	52
3.3.3 文件系统操作	53
3.3.4 信号处理	54
3.3.5 网络通信 (Socket)	55
3.3.6 时间管理	55
3.3.7 系统信息与控制	56
3.3.8 其他重要系统调用	56
第四章 总结与展望	57
4.1 工作总结	57
4.2 经验总结	57
4.3 未来计划	58
附录 A 一些重要的类的具体字段解析	59

第一章 概述

1.1 F7LY 内核的介绍

F7LY-OS 是 C++ 编写的支持 RISC-V 与 LoongArch 双架构的宏内核模块化操作系统，由来自武汉大学的三名队员使用 C++ 从 0 参考 xv6 并改编、增加功能构建而成。使用分体的硬件抽象层实现架构无关的内核核心功能，进行多阶段启动，提供统一的进程管理，细粒度的内存资源动态管理，拥有统一接口的虚拟文件系统，可使用 C++ 的多态特性支持 fat32 与 ext4，以及基础的信号机制。开发过程的全部 commit 记录请参见原仓库地址 [KidsZZ/F7LY-OS]。^①

F7LY 使用模块化、面向对象的方式组织内核代码，将宏内核各个部分独立分成不同的模块。通过明确模块之间的接口，减少模块间的相互依赖和干扰，从而使内核代码复用成为可能，目标是提供功能与性能兼备的模块以及可读性强、可迁移性强的代码。

F7LY 致力于实现高质量的代码，使内核充分发挥 C++ 编译型语言的优点，接近硬件层的代码书写与直接操作内存的特性可更直观地操作内核中的资源管理。功能方面，实现了完整的 C++ ABI 支持，包括异常处理机制，支持标准 POSIX 系统调用；性能方面，通过懒分配和写时复制优化 execve 执行速度，使用页缓存加速文件读写，减少 I/O 访问次数。

F7LY 严格遵循 RISC-V 与 LoongArch 的硬件标准与中断处理规范，使用统一的 trap 管理框架支持 RISC-V PLIC 和 LoongArch 的 PLIC/ExtIOI 中断控制器，来正确处理系统调用和各类例外与中断。系统调用严格按照 System Calls Manual 手册进行开发，确保实现的所有系统调用符合 Linux raw syscall 规范，为运行在 F7LY 的用户态程序提供了完备可靠的系统调用支持。

1.2 主要工作

1.2.1 参考的代码

在 F7LY 项目的内核部分，我们对往年参赛团队的成果以及成熟的第三方库进行了深度移植与优化，以确保项目的高效性与可靠性。

- XN6: 在 LoongArch 架构的支持方面，页表管理及硬件抽象层设计参考了 2024 年武汉大学团队的 [XN6]^②项目，实现了对 LoongArch 架构的高效适配。

^① <https://github.com/KidsZZ/F7LY-OS>

^② <https://gitlab.eduxiji.net/T202410486992576/OSKernel2024-2k1000la-xv6>

- **xv6-2021**: 异常处理机制及内核地址布局的设计, F7LY 借鉴了 MIT S081 课程的 [xv6-2021]^①, 并移植了其物理内存管理的伙伴分配器算法, 以提升内存分配的效率和稳定性。
- **EASTL**: 在 C++ 代码构建方面, 由于内核环境不支持标准 C++ 库 (std), 我们引入了 EASTL 库, 以支持部分模块对高效数据结构的需求。
- **liballoc**: 动态堆内存管理采用了 Liballoc 分配器库, 经过集成与优化, 确保了内核中内存分配的高效性与可扩展性。
- **lwext4**: 文件系统部分, 我们对原本为 C 语言编写的 lwext4 库进行了适配与重构, 以满足内核环境对高性能文件操作的需求。
- **Open-NPStack**: 网络栈则结合了 VirtIO 网络驱动与 [ONPS 网络栈]^②, 确保了在不同硬件平台上的网络通信能力。
- **oskernel-AVX**: 移植自 VisionFive2 开发板上的 sd 卡驱动参考了 2023 年 [SQL 巨佬]^③ 队伍的实现, 确保了对 SD 卡的高效访问。

1.2.2 我们的工作

F7LY-OS 的内核部分由三名成员共同完成, 在参考前人工作的基础上, 完全从零开始构建, 团队成员经过多轮优化与完善, 截至目前已完成超过 700 次的代码提交。

在初赛阶段, F7LY 成功实现了 LoongArch 和 RISC-V 两种架构下的机器启动、中断处理、内存管理、进程调度及文件系统框架搭建与功能实现, 并且成功实现了简易的信号处理机制。经过多轮测试验证, 双架构均通过了初赛的 basic 测试、busybox 测试、libctest 大部分测试、libcbench 测试和 lua 测试。此外, 我们已完成 60 余个系统调用的功能实现, 并且确保了这些调用在两种架构下的正确性。

在决赛阶段, F7LY 的工作进一步扩展并深化, 主要工作包括但不限于以下几个方面:

- **内存管理**: 对内存布局进行了重大调整, 修改了堆内存及物理内存的范围, 并添加了共享内存区域。通过使用程序段的方法重新规划进程的地址空间, 使得进程内存区域的管理更加科学合理。
- **进程管理**: 重构了进程管理模块, 根据进程的具体功能对其内存管理进行重新划分, 使得进程内存管理更加清晰且模块化。新增了进程内存管理器, 进一步

① <https://github.com/mit-pdos/xv6-public>

② <https://gitee.com/Neo-T/open-npstack>

③ <https://gitlab.eduxiji.net/202310487101114/oskernel2023-avx>

提升了对进程内存的管理效率。

- **文件系统：**我们移除了之前功能不全的 C++ 文件系统，成功移植了成熟的 lwext4 文件系统，并建立了 VFS 层来转发文件操作请求，确保了系统文件操作的可扩展性。
- **进程间通信：**构建了更加完整和强大的进程间通信机制，包括完善的信号机制、共享内存、匿名管道和有名管道、跨主机 Socket 通信等，为内核提供了更高效的进程间数据交换能力。
- **线程管理：**新增了线程支持，结合线程 ID 等字段实现了线程级管理，使得多线程操作在内核中得以顺畅执行。
- **系统调用完善：**在 LTP 测试过程中，依据 Linux 手册要求对系统调用进行进一步的错误处理与功能完善，至今已成功实现并验证了 210 余个系统调用。
- **硬件移植：**我们成功尝试移植 F7LY 内核至 VisionFive2 开发板，并在物理硬件环境中实现了内核的正确初始化与跳转至用户态。通过完善的 SD 卡驱动，我们成功实现了对硬盘存储数据的读写操作。

通过这些关键性的进展与优化，F7LY 内核不仅在多架构、多模块的实现上实现了突破，同时也为未来的扩展与优化奠定了坚实的基础。

1.3 整体架构

kernel 目录中对代码进行了模块化分区，支持双架构设计。

- boot 系统启动模块 - 负责内核的启动流程，包含 RISC-V 和 LoongArch 两种架构的启动代码，实现从 bootloader 到内核 main 函数的跳转，初始化基础硬件环境。
- devs 设备管理模块 - 实现统一的设备抽象层，包含字符设备、块设备、流设备等抽象类，提供设备管理器进行设备注册和管理，支持 UART、控制台、磁盘等硬件设备。
- fs 文件系统模块 - 实现了 VFS（虚拟文件系统）层，支持多种文件系统包括 ext4、FAT、ramfs 等，提供统一的文件操作接口，包含缓冲区管理、目录项缓存、inode 管理等核心组件。
- hal 硬件抽象层 - 提供跨架构的硬件抽象，封装 CPU 相关操作和上下文切换，为上层模块提供统一的硬件接口，支持 RISC-V 和 LoongArch 双架构。
- libs 内核库模块 - 提供内核所需的基础库函数，包括字符串操作、打印输出、内存分配器、模板算法、C++ 运行时支持等，为内核其他模块提供基础服务。
- mem 内存管理模块 - 实现完整的内存管理体系，包含物理内存管理器（支持伙

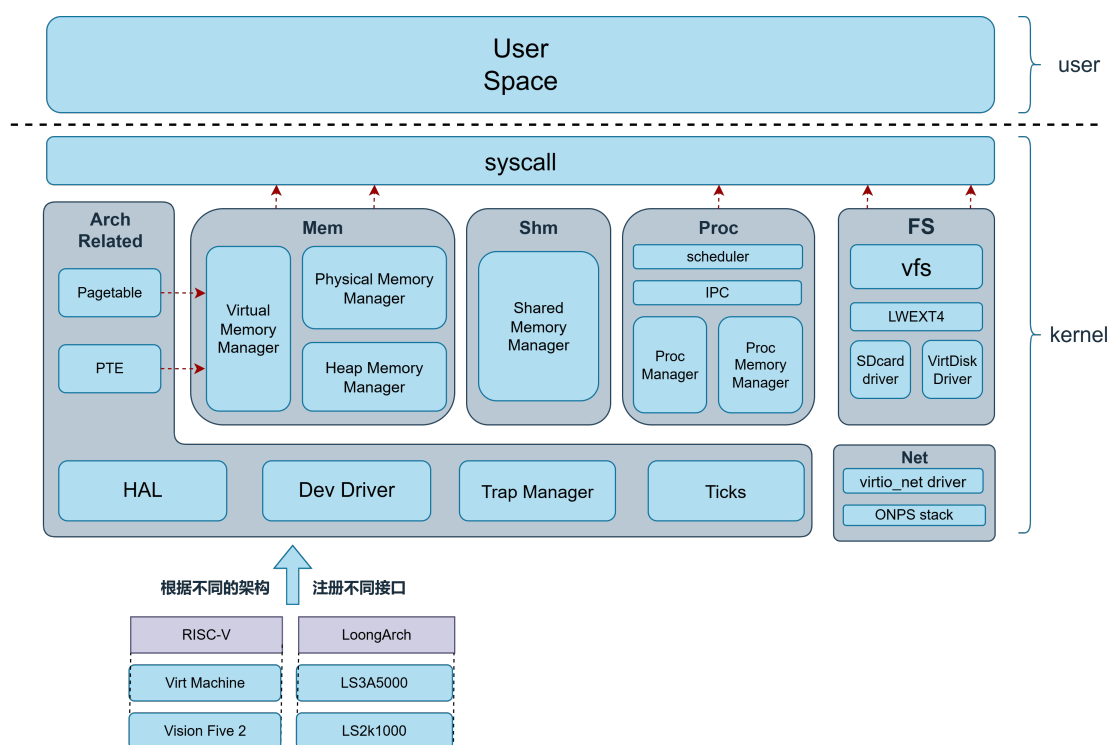


图 1-1 整体架构

伴系统算法)、虚拟内存管理器、堆内存管理器、slab 分配器等，提供高效的内存分配和回收机制。

- **shm 共享内存模块** - 实现进程间共享内存机制，支持匿名共享内存和有名共享内存，提供高效的跨进程数据交换能力，包含共享内存分配、映射、同步等功能。
- **proc 进程管理模块** - 实现进程创建、调度、同步等核心功能，包含进程管理器、调度器、信号处理、管道通信、锁机制等，支持多进程并发执行。
- **sys 系统调用模块** - 实现系统调用处理机制，提供用户态程序与内核的交互接口，包含系统调用分发、参数传递、权限检查等功能。
- **trap 中断与异常处理模块** - 处理硬件中断、异常和系统调用，实现中断向量表设置、异常处理流程、时钟中断管理等，支持双架构的不同中断机制。
- **tm 时间管理模块** - 提供时间和定时器相关服务，实现定时器管理器、时间接口等，为进程调度和系统调用提供时间基础服务。
- **net 网络模块** - 网络协议栈的基础框架以及 VirtIO-Netdriver。

```
F7LY-OS
├── build                //工程构建文件
├── kernel              //内核文件
│   ├── boot            //内核入口以及主函数
│   ├── devs            //设备驱动
│   ├── fs              //文件系统
│   │   ├── drivers     //硬盘等设备驱动
│   │   ├── lwext4       //lwext4库
│   │   ├── linux        //linux相关定义
│   │   └── vfs          //虚拟文件系统
│   │       └── file      //用于继承的统一文件操作接口
│   ├── hal            //与平台相关的寄存器定义
│   ├── libs            //自定义的库操作支持
│   ├── link            //链接脚本
│   ├── mem             //内存管理
│   ├── shm             //共享内存
│   ├── net             //网络相关
│   ├── proc            //进程调度与管理
│   ├── sys             //系统调用
│   ├── tm              //时钟
│   └── trap            //中断处理
├── thirdparty          //使用的第三方库
│   └── EASTL            //EASTL库
├── user                //用户程序
└── Makefile
```

第二章 内核详细介绍

2.1 机器启动

F7LY OS 支持双架构启动，机器启动的源文件分别在 `kernel/boot/riscv/` 和 `kernel/boot/loongarch/` 目录下。

启动流程遵循经典的多阶段启动模式：`Bootloader -> entry.S -> start.cc -> main.cc`

整个 `kernel` 可执行文件的入口点在各自架构的 `entry.S` 中的 `_entry` 标签，通过链接脚本指定。

RISC-V 架构：基于 SBI(Supervisor Binary Interface) 规范，Bootloader 工作在 M-mode，内核代码从 `entry.S` 开始运行在 S-mode。

LoongArch 架构：采用 LoongArch 原生启动方式，通过 ACPI 等标准接口进行硬件初始化。

2.1.1 RISC-V 启动流程

2.1.1.1 entry.S 阶段

- 负责设置操作系统的栈指针 `sp` 到合适位置，为后续 C++ 代码执行做准备。
- 每个 CPU 核心分配 4KB 独立栈空间，通过 `hartid` 进行区分。
- 完成栈设置后跳转到 `start()` 函数。

```
_entry:
    la sp, stack0           # Load base addr of stack
    li t0, 1024*4           # 4KB space per stack
    mv t1, a0               # gain hartid
    addi t1, t1, 1
    mul t0, t0, t1           # cal current offset of stack
    add sp, sp, t0           # set stack pointer
    call start              # jump to start func
```

2.1.1.2 start.cc 阶段

关键寄存器处理：

- `a0`：存储硬件线程编号 `hartid`。
- `a1`：设备树地址信息 `dtb_entry`。
- `sp`：已在 `entry.S` 中设置完成。

主要工作：

- 关闭分页机制，使用物理地址访问
- 设置临时 trap 处理函数为死循环
- 将 hartid 保存到 tp 寄存器供后续使用
- 调用 main() 函数进行系统初始化

```
void start(uint64 hartid, uint64 dtb_entry)
{
    riscv::csr::_write_csr_(riscv::csr::satp, 0);
    riscv::csr::_write_csr_(riscv::csr::stvec,
        ↪ (uint64)trap_loop);
    riscv::w_tp(hartid);    // Save hartid
    main();
}
```

2.1.1.3 主函数初始化

系统初始化分为三个主要阶段：

1. 基础服务初始化

```
k_printer.init();
trap_mgr.init();
plic_mgr.init();
```

2. 内存管理初始化

```
mem::k_pmm.init();
mem::k_vmm.init();
mem::k_hmm.init();
```

3. 进程和设备管理

```
proc::k_pm.init();
dev::k_devm.register_stdin();
riscv::qemu::disk_driver.init();
syscall::k_syscall_handler.init();
```

4. 启动用户进程和调度器

```
proc::k_pm.user_init();
proc::k_scheduler.start_schedule();
```

2.1.2 LoongArch 启动流程

loongarch 部分将 `start.cc` 阶段合并到 `entry.S` 中，在其中完成设置设备操作空间、设置指令数据访问空间，CSR 寄存器的初始化，以及栈空间的设定，并开启 fpu 浮点数指令。

entry 只对每个 CPU 设定自己的栈空间，然后跳转到 main 函数

```
li.d      $t0, 0x8000000000000001
csrwr     $t0, LOONGARCH_CSR_DMWIN0 # 设置设备操作空间
li.d      $t0, (0x9000000000000001 | 1 << 4)
csrwr     $t0, LOONGARCH_CSR_DMWIN1 # 设置指令数据访问空间
csrwr     $t0, LOONGARCH_CSR_TLBRENTY

li.w      $t0, 0xb0                # PLV=0, IE=0, PG=1
csrwr     $t0, LOONGARCH_CSR_CRMD
li.w      $t0, 0x00                # PPLV=0, PIE=0, PWE=0
csrwr     $t0, LOONGARCH_CSR_PRMD
li.w      $t0, 0x01                # FPE=1, SXE=0, ASXE=0, BTE=0
csrwr     $t0, LOONGARCH_CSR_EUEN
```

其余大部分启动的基本流程与 riscv 类似，在中断与虚拟化方面 LoongArch 架构采用不同的启动方式：

- 使用 ACPI 控制器进行硬件发现和初始化。
- 通过 APIC 和 ExtIOI 进行中断管理。
- 支持 LoongArch 原生的虚拟化和安全特性。

```
void main()
{
    k_printer.init();
    dev::acpi::k_acpi_controller.init();
    apic_init();
    extioi_init();
    // ... 后续与 RISC-V 类似的初始化流程
```

```
}
```

2.1.3 双架构支持特色

- **统一的抽象层**：通过 HAL(硬件抽象层) 实现跨架构代码复用。
- **架构特定优化**：针对不同架构的特性进行专门优化。
- **模块化设计**：核心模块与架构相关代码分离，便于维护和扩展。
- **现代化实现**：采用 C++ 面向对象设计，提供类型安全和更好的代码组织。

2.1.3.1 启动完成标志

系统启动完成后会显示欢迎信息并启动调度器：

```
=== SYSTEM BOOT COMPLETE ===
Kernel space successfully initialized
```

此时内核已完成所有初始化工作，进入正常的多进程调度运行状态。

2.2 中断管理器

从用户态到内核态的切换需要中断和异常的频繁处理，此处 F7LY 在迁移 xv6 代码的过程中选择了对象化中断管理器类 `TrapManager`，并实例化全局对象 `trap_mgr`。

```
class trap_manager
{
public:
    void init();
    void inithart();
    int devintr();          // 处理外部中断和软件中断
    void usertrap();
    void usertrapret();
    void machine_trap();
    void kerneltrap();
private:
    void timertick();
    SpinLock tickslock;
    uint ticks;
    uint timeslice;
```

```
};
```

F7LY 在其底层根据 RISC-V 和 loongarch 的不同硬件设备使用而封装了不同的中断处理器，在不同文件夹下实现了 PLIC、EXTIOI、APIC 的驱动并应用在中断处理之中。

- RISC-V 在处理中设定了中断处理的入口 (`uservec`) 以及返回的入口 (`user-trapret`), 具体逻辑参照了 xv6 的中断处理, 但封装在对象之中, 并使用包装函数 (`wrap_` 前缀的全局函数) 设置中断入口。
- Loongarch 针对架构的特殊性设置了 TLB 重填的处理程序入口 (`tlbrefill.S`), 以及机器异常的处理程序 (`merror.S`), 使用类似的逻辑进行接口统一化, 使得上层程序可以统一调用。

如果用户程序发出系统调用, 或者做了一些非法的事情, 或者设备中断, 那么在用户空间中执行时就可能会产生陷阱。来自用户空间的陷阱的高级路径是 `uservec`, 然后是 `usertrap`; 返回时, 先是 `usertrapret`, 然后是 `userret`。

首先通过 `crmd` 寄存器确认陷阱确实来自用户模式, 然后将中断入口点 (`eentry`) 设为 `kernelvec`, 并获取当前进程信息并保存用户程序计数器。

函数通过检查异常代码 (`CSR_ESTAT_ECODE`) 来区分不同类型的陷阱:

- **系统调用 (0xb):**
 - 检查进程是否被标记为结束。
 - 调整返回地址, 使其指向 `ecall` 指令之后的指令。
 - 启用中断, 然后调用系统调用处理器。
- **内存错误 (0x1 或 0x2):**
 - 调用 `mmap_handler` 来处理缺页异常。
 - 如果处理失败, 则标记进程为已终止。
- **设备中断:**
 - 调用 `devintr()` 处理设备中断。
- **未知陷阱:**
 - 打印错误信息并标记进程为已终止。

如果是时钟中断 (`which_dev == 2`), 则增加时间片计数, 当达到阈值 (10 个时间片) 时调用 `yield`。最后调用 `usertrapret()` 函数返回用户态, 该函数负责恢复用户态执行环境。

2.3 内存管理

2.3.1 地址空间

2.3.1.1 内核链接地址

F7LY 在支持多架构内核时，并未采用“一体适配”的简化策略，而是根据架构自身特性量身定制内核布局，以保障启动路径的最小依赖性和最大清晰性。

对于 RISC-V，qemu 的入口地址位于 `0x8020_0000`。在此处，F7LY 采用 OPENSBI 自动识别内核的 ELF 文件入口地址（LMA），低地址采用直接映射的方式映射所有 MMIO 设备以及内核代码。因为在进入到 `entry.S` 时仍然未开启分页，所以映射时低处地址采用直接映射便于访问。

与 RISC-V 使用 QEMU 默认跳转地址（`0x80200000`）不同，LoongArch 架构的启动顺序允许我们将内核加载地址与运行地址（VMA = LMA）分离，从而更灵活地构造分页映射策略。F7LY 的 LoongArch 端应用 loongarch 的直接映射窗口，在链接脚本处使用 `0x90000000080000000` 加载内核镜像。

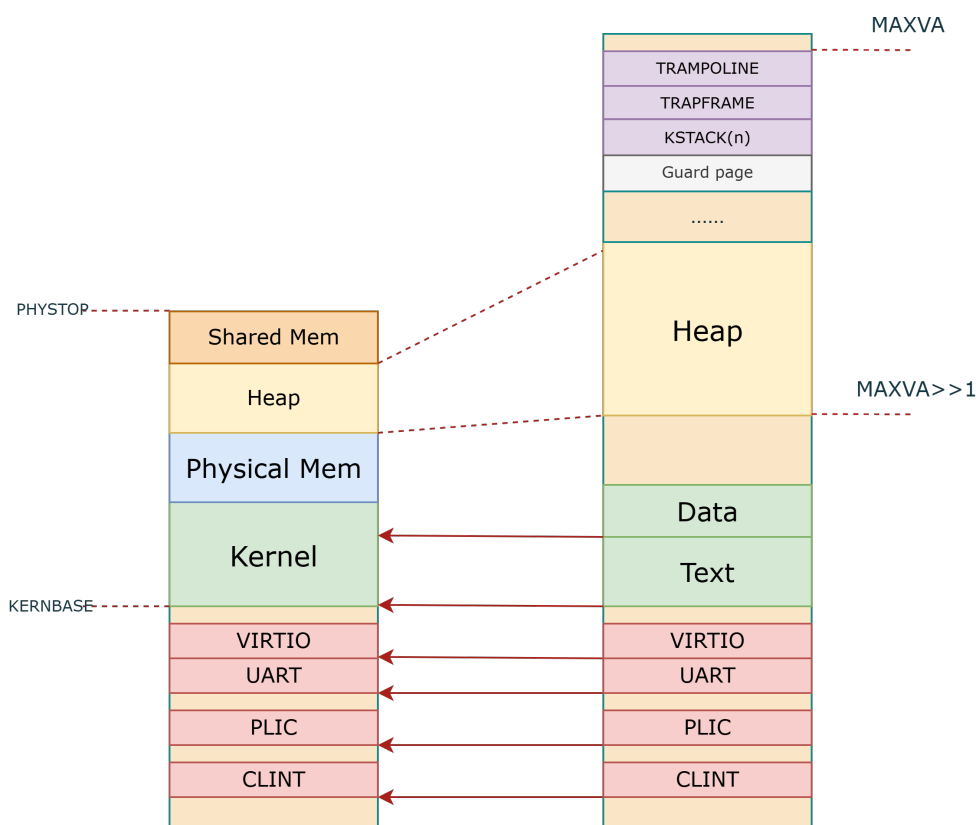


图 2-1 内核地址空间布局

2.3.1.2 地址空间布局

F7LY 的地址空间的设计参考了 xv6 的物理地址布局，内核态页表保存在全部内核地址空间 Kernel 中，其记录了内存的空间布局。用户页表在用户空间中记录用户态地址。

对于内核地址空间，为方便管理物理地址，从 KERNBASE 开始映射内核映像；堆内存从地址 `0x84000000` 开始，剩余的空闲物理地址交由物理内存管理器管理，用于后续的动态分配。

对于用户地址空间，为便于堆空间的增长和用户程序的动态访问，将用户地址的代码从 `Proc_base` 进行加载。每当用户对内存进行申请时，采用对 `p->_heaptop` 指针的移动进行扩张。由于 loongarch 的加载地址与 riscv 不一致，我们在 `execve` 对程序进行加载时显式设置程序的空间开始地址（`PROC_BASE`），并从此处开始设置 `p->_heaptop` 指针，装载完成后的指针位置既标志了进程此时的大小，又可作为堆顶指针开始动态内存分配。

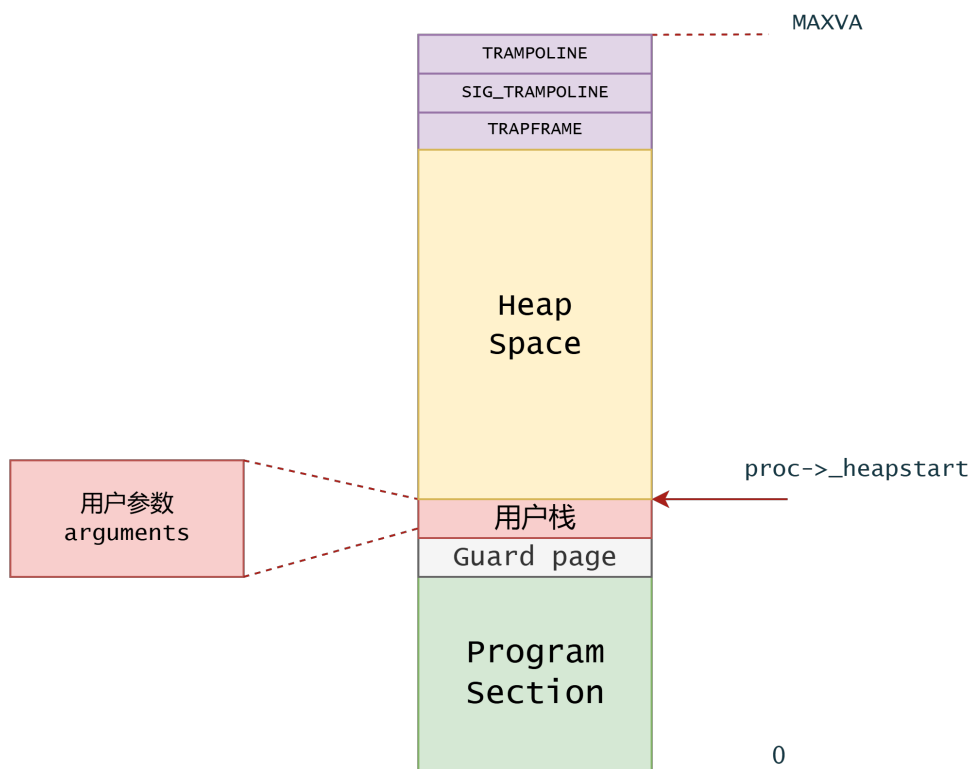


图 2-2 用户地址空间布局

2.3.2 物理内存管理

2.3.2.1 内核动态内存分配器

F7LY 使用粗细粒度混合分配管理内核所需的物理内存，统一使用伙伴内存分配器进行粗粒度分配。伙伴分配器用于管理一片连续内存，从一个地址开始初始化，并设置最多支持的分配粒度便可以指定伙伴分配器管理的一片连续内存。

```
#define PAGE_ORDER 10
#define PGNUM (1 << 15)
#define BSSIZE 10 //最多支持 2^10 = 1024 页的分配粒度

class BuddySystem {
public:
    int Alloc(int size);           //按照大小分配
    void Free(int offset);
    void* alloc_pages(int count); //按照页数分配
    void free_pages(void* ptr);
private:
    uint8* tree;                  //建立对一片内存管理的树结构
    uint8* base_ptr;              //管理内存的起始地址
}
```

伙伴分配器 (Buddy Allocator) 通过分配和管理内存块来满足不同大小的内存请求，并进行高效的合并和分割操作。其优点在于分配和释放内存块的操作非常快速，且通过内存块大小的选择和合并操作，有效减少了外部碎片。但是对于页面内部的空间碎片，需要更细粒度的管理器进行分配，考虑到空闲物理空间与内核堆空间应用场景的不同，我们选择使用不同的细粒度分配器进行管理。

2.3.2.2 内核物理内存

内核中的物理空间，即实际分配的物理页面是在操作系统中应用最多的空间，F7LY 使用对象化资源管理的思想，设计物理管理类 `PhysicalMemoryManager` 并创建全局对象 `k_pmm` 进行管理。

```
class PhysicalMemoryManager
{
public:
```

```

static void *alloc_page();
static void free_page(void *pa);
static void *kmalloc(size_t size);
static void *kcalloc(uint n, size_t size);
void clear_page(void *pa);
private:
    static BuddySystem *_buddy;
    static uint64 pa_start;
    // .....
};
extern PhysicalMemoryManager k_pmm;

```

`k_pmm` 在初始化时会使用地址布局中的地址标志 (`pa_start`)，原本 Buddy 的初始化放在这里，Buddy 变成 pmm 的一个成员 `pa_start` 是 buddy 系统在物理内存中的起始地址，加上一个 `Sizeof(BuddySystem)` 后后面存的东西是 tree，然后 tree 存完了之后才是 buddy 系统管理的那块内存。加上的 BSSIZE 是预留来放 BuddySystem 的大小和 tree 的大小，这之后才是 buddy 系统管理的那块内存，这时 `pa_start` 指向的就是 buddy 系统管理的那块内存的开始地址，再被初始化为 buddy 的基址。

```

void PhysicalMemoryManager::init()
{
    memlock.init("memlock");
    pa_start = reinterpret_cast<uint64_t>(end);
    pa_start = (pa_start + PGSIZE - 1) & ~(PGSIZE - 1);
    _buddy = reinterpret_cast<BuddySystem*>(pa_start);
    pa_start += BSSIZE * PGSIZE;
    memset(_buddy, 0, BSSIZE * PGSIZE);
    _buddy->Initialize(pa_start);
}

```

对于更细粒度的大小分配（函数 `kmalloc`），F7LY 采用了 linux 的 `SlabAllocator`。slab allocator 背后的思想是缓存经常使用的 object 并保持在初始状态供 kernel 使用。如果被基于 object 的 allocator，内核将耗费很多时间在分配，初始化和释放相同的 object。slab allocator 的目的就是缓存一些被释放的 object 因此这些基础的 structures 在多次调用期间被预留起来。

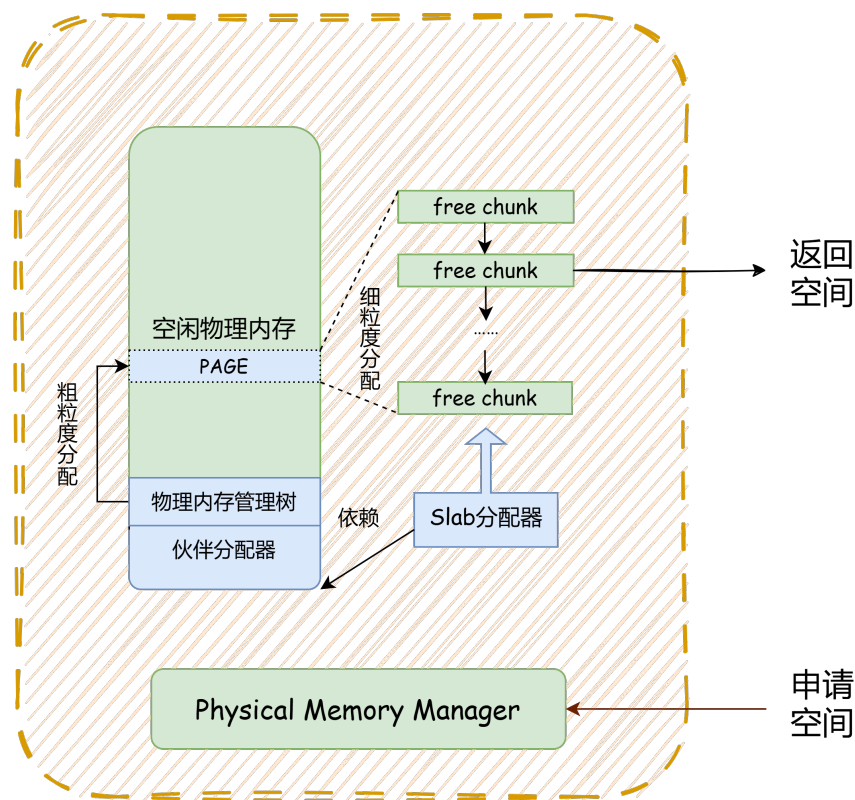


图 2-3 物理内存管理

slab allocator 由一组 cache 组成，这些 cache 由一个叫做 cache chain 的双向循环链表连接在一起。在 slab allocator 的上下文文件，一个 cache 就是一个管理许多像 `mm_struct` 或者 `fs_cache` 这种特殊类型的 object 的管理者。这些 cache 通过 cache struct 的 next 字段连接在一起。

每一 cache 维护了有多个连续物理 page 组成的 block，这些 block 称之为 slab。slab 被切分成很多小块来存放 slab 自身的数据结构和其管理的 object。

```
class SlabCache
{
private:
    static constexpr uint32
        ↪ DEFAULT_MAX_FREE_SLABS_ALLOWED = 5;
    uint32 obj_size_;
    uint32 free_slabs_count_;
    LinkedList<Slab> free_slabs_;
    LinkedList<Slab> partial_slabs_;
    LinkedList<Slab> full_slabs_;
}
```

```

}
.....

```

Slab 与 Buddy 二者分层次调用即可实现细粒度的物理内存分配，更小块的内存分配可以帮忙消除 buddy allocator 原本会造成的内部碎片问题。

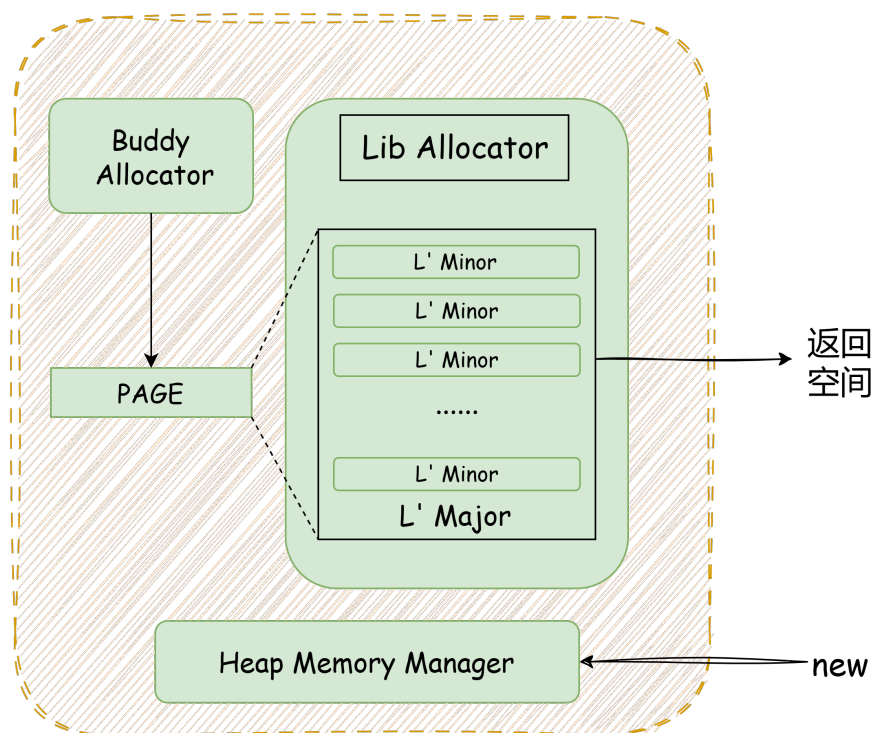


图 2-4 堆空间管理

2.3.2.3 内核堆内存分配

与物理内存类似，堆内存是 `cpp` 中用 `new` 动态创建对象的重要方法，为在内核环境中实现 `new` 与 `delete` 的正确功能，我们在内核中开辟一块堆空间，并重载 `new` 与 `delete` 来完成我们自己的对象实例化。

```

class HeapMemoryManager
{
private:
    SpinLock _lock;
    BuddySystem* _k_allocator_coarse;
    L_Allocator _k_allocator_fine;
public:

```

```

HeapMemoryManager() {};
void init(const char *lock_name, uint64_t
    ↪ heap_start);
void *allocate(uint64 size);
void free(void *p);
};
extern HeapMemoryManager k_hmm;

```

对于堆，设置管理类 `HeapMemoryManager` 并创建全局对象 `k_hmm` 进行管理。`k_hmm` 在初始化时会从堆地址开始初始化 Buddy System，并将此处划分为堆空间。

为提升堆分配的性能、减少碎片，我们在堆空间上引入了第三方高效内存分配器 `LibAllocator`^①，主要用于进行高效内存管理。分配器设计为二层结构：上层是**粗粒度分配器**（`L'Major`），负责大块内存获取；下层是**细粒度分配器**（`L'Minor`），用于将大块切分成适用于常规 `new / malloc` 调用的空间，极大提升了小对象分配的效率。

F7LY 对 `LibAllocator` 进行了定制性适配，使其不再从物理内存中直接申请页帧，而是通过内核内部的 `BuddyAllocator` 分配大块内存作为 `L'Major` 的来源。这种结构层次分明、责任明确，既保持了 `Buddy` 分配器的可控性，也引入了 `LibAllocator` 在细粒度分配上的高效策略。

F7LY 重载了标准的 `new / delete` 运算符，使其默认在堆空间上分配内存并交由 `k_hmm` 管理：

```

void * operator new(uint64 size)
{
    void *p = mem::k_hmm.allocate(size);
    return p;
}

void operator delete(void * p) noexcept
{
    mem::k_hmm.free(p);
}
// 省略展示其余用法的 new 和 delete 运算符。

```

① <https://github.com/blanham/liballoc>

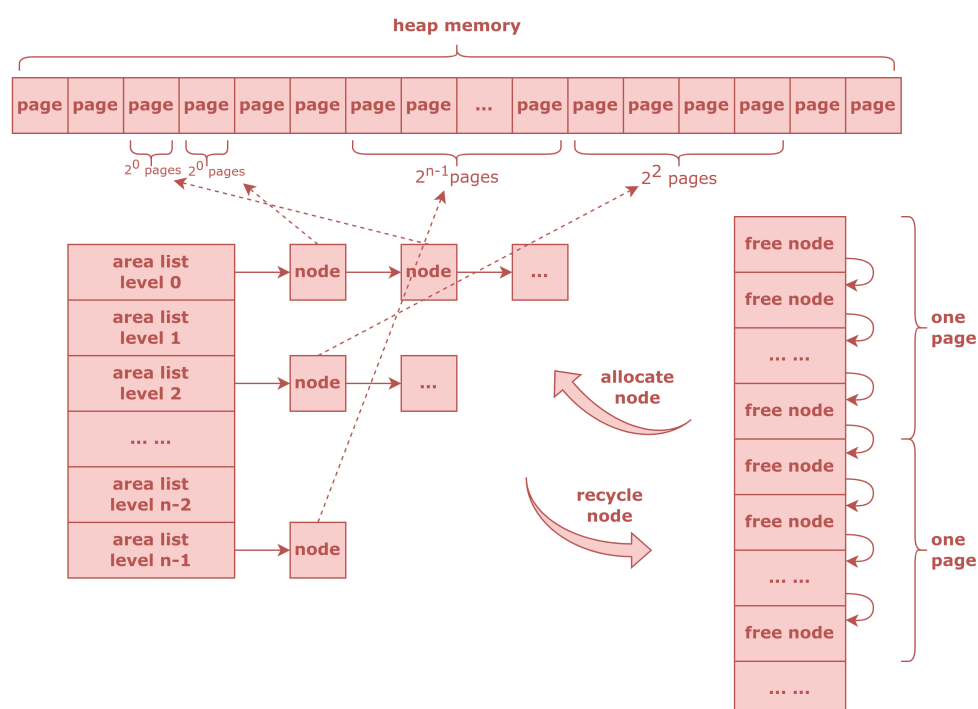


图 2-5 Buddy Allocator 分配示意图

通过这种方式，内核中的对象创建和销毁行为得以统一，所有堆内分配操作均通过 `k_hmm` 路由，有效避免了裸指针操作和分配器混用的问题。

2.3.2.4 地址空间管理

F7LY 采用**静态预配置 + 管理器分派**的机制进行地址空间管理。核心由 `VirtualMemoryManager` 管理器统一负责虚拟地址的分配、映射与回收，底层则由 `PageTable` 类协助完成页表操作与维护。这种结构设计不仅提升了内存管理的确定性和效率，也使得地址空间的生命周期管理更为清晰。

针对 `mmap` 所涉及的文件映射与匿名映射，F7LY 在每个进程控制块（PCB）中**预先分配了一块 VMA 表区域**，用于记录与管理该进程的虚拟内存区域（VMA），避免了运行时动态结构分配所带来的复杂性与不确定性。

页表 `PageTable` 类用于抽象和管理多级页表结构。其核心成员变量包括：

- `base_addr` 页表的物理基地址，指向页表的起始位置。
- `_is_global` 标记该页表是否为全局页表（如内核页表）。

主要接口包括：

- `walk(uint64 va, bool alloc)`：软件递归遍历多级页表，查找或分配虚拟地

址 `va` 对应的 PTE。可用于递归查找虚拟地址的映射关系，必要时分配中间页表。

- `walk_addr(uint64 va)` : 返回虚拟地址 `va` 映射的物理地址指针。
- `freewalk()` / `freewalk_mapped()` : 递归释放页表及其映射的物理页。
- `get_pte(index)` / `set_pte(index, pte)` : 获取/设置指定索引的 PTE。

在 `walk` 函数中，由于 RISC-V 使用 SV39 标准页表，而 loongarch 使用 4 级页表，二者不可统一，在此处 F7LY 分别定义了不同的实现，并在编译时根据宏进行区别。

PTE PTE 类封装了单个页表项 (Page Table Entry)，通常包含如下信息：

- 物理页帧号 (PPN)。
- 有效位 (Valid)、读/写/执行权限 (R/W/X)、用户/超级用户位 (U/S) 等标志位。

上述信息由于 loongarch 和 RISC-V 的架构不同而设置了不同的访问接口，可用于映射时设置或访问时查询。

虚拟内存管理器 VirtualMemoryManager 是内核虚拟内存的管理器，同样设置了全局对象 `k_vmm` 进行内存管理，它为进程和内核提供统一的虚拟地址空间分配、映射、回收、拷贝等功能，屏蔽了底层不同架构（如 RISC-V、LoongArch）页表实现的差异。

主要功能如下：

- **虚拟地址到物理地址的映射与解除映射**：通过 `map_pages`、`vmunmap`、`kvmmap` 等接口，将虚拟地址空间映射到物理内存，或解除映射关系。
- **内核页表的创建与初始化**：通过 `kvmmake` 创建内核页表，`kvmmap` 完成内核空间的初始映射。
- **用户空间内存分配与回收**：通过 `uvmmalloc`、`uvmdealloc` 等接口，支持进程虚拟空间的动态增长与收缩。
- **内存拷贝与字符串拷贝**：提供 `copy_in`、`copy_out`、`copy_str_in` 等接口，实现用户态与内核态之间的数据安全拷贝。

通过条件编译（如 `#ifdef RISCV`）自动适配不同架构下的页表实现，保证接口一致性与可移植性。

文件映射与匿名映射 针对 `mmap` 系统调用的匿名映射和文件映射，F7LY 采用在进程控制块中预留一片区域的方法进行管理。

```
struct vma
{
    int used;
    uint64 addr;
    int len;
    int prot;
    int flags;
    int vfd;
    fs::normal_file *vfile;
    int offset;
    uint64 max_len;
    bool is_expandable;
};
```

当用户态调用 `mmap` 系统调用时，内核会查找该进程 PCB 中的空闲 VMA 槽位，将映射信息记录其中，并返回分配好的虚拟地址。若为文件映射，则填入对应的 `vfile` 和 `offset` 字段；若为匿名映射，则标记 `flags` 中的 `MAP_ANONYMOUS`，并将 `vfile` 设为 `nullptr`。

2.3.2.5 缺页异常处理

F7LY 目前能够利用缺页异常处理来实现写时复制（Copy on write）、地址空间的懒分配（Lazy page allocation）以及用户的地址检查机制。

当用户程序因缺页异常进入内核时，两个架构的异常处理程序使用同样的处理逻辑，先检查缺页的地址是否处于物理空间，或处于 `vma` 记录的地址空间内，若是，则分配物理页面并建立映射。若不是，则抛出缺页错误。

2.4 进程管理

2.4.1 进程控制块 (PCB)

进程是操作系统中资源分配的基本单位。每个进程都有自己独立的地址空间和资源，如内存、文件描述符等。线程是操作系统中 CPU 调度的基本单位，线程共享所在的进程的地址空间和资源，但是有独立的执行上下文。

许多操作系统内核的设计都将进程和线程分开设计，分别使用 `Process` 和

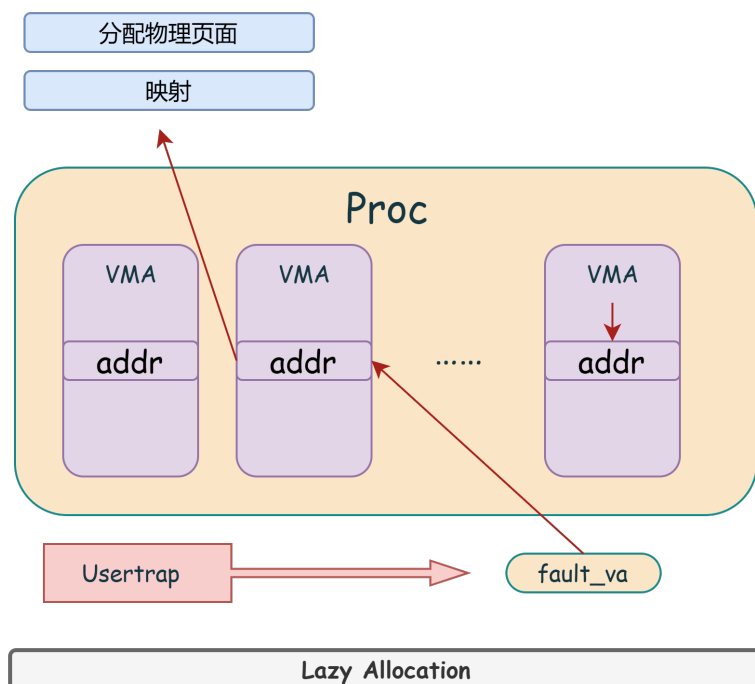


图 2-6 缺页异常处理流程

Thread 结构体来表示。但其实在 Linux 内核中并没有严格地区分进程和线程，而是通过一组统一的 API 来操作任务。例如 `sys_clone` 通过不同的 flags 组合创建共享不同资源的新任务，因此进程和线程的创建本质上是类似的。F7LY 此处统一使用 Pcb 结构体，并在 Pcb 中设置 `globa_id`、`pid`、`tid`、`ppid`、`pgid` 等标准 Linux 进程标识符用于标记进程和线程状态，线程的共享内存状况也一并记录便于管理。

2.4.1.1 PCB 的组成

```
class Pcb;
extern Pcb k_proc_pool[num_process];
```

详细字段放于文末附录进行介绍。

2.4.1.2 进程的状态

在所有的使用之中，进程需要的状态可简略分为 5 种，含义如下：

进程状态的切换如下：

- 创建进程池时构造函数中赋值 `UNUSED`
- `alloc_proc()` 从空闲进程中初始化时创建用户页表分配物理空间，并切换为

表 2-1 进程状态说明

状态	含义
RUNNING	正在运行或准备运行的任务。此状态下，任务占用 CPU，执行其代码
RUNNABLE	进程处于就绪队列中，当正在运行的进程放弃 CPU 或时间片耗尽后会按照队列获取 CPU
UNUSED	进程创建而来的状态，即初始状态
USED	进程分配时进行切换，在创建和就绪之间的状态
SLEEPING	时间片耗尽或等待某个事件。该状态下进程可以被信号唤醒
ZOMBIE	进程已终止，但 PCB 依然存在，以便父进程读取退出状态并等待回收

USED。

- `user_init()` 中初始化进程，并在所有初始化完成后切换为 RUNNABLE。
- 调度器从就绪队列中找到优先级最高的进程，开始运行，并将状态设为 RUNNING。
- 当被系统调用终止或时间片用尽时进程放弃 CPU，状态回到 RUNNABLE。
- 当 `futex` 或 `sleep` 系统调用让进程进入休眠时，进程切换为 SLEEPING，此时等待信号唤醒进程。
- 进程使用 `exit` 退出时状态切换为 ZOMBIE。

2.4.2 进程调度

为便于全局调用，F7LY 使用调度器 Scheduler 类以及全局对象 `k_scheduler` 进行进程调度管理，其中具体的调度方法使用了与 MIT xv6 类似的逻辑方法。

```
class Scheduler
{
private:
    SpinLock _sche_lock;
public:
    Scheduler() = default;
    void init(const char *name);
    void switch_to_proc(Pcb* p);    // 调度切换
    int get_highest_proirity();
    void start_schedule();
```

```

void yield(); // 进程放弃 CPU，状
    ↪ 态从 RUNNING 改为 RUNNABLE
void call_sched();
};

```

2.4.2.1 上下文切换

汇编函数 `swtch.S` 用于管理一个到旧进程内核线程的用户-内核转换（系统调用或中断），一个到当前 CPU 调度程序线程的上下文切换，一个到新进程内核线程的上下文切换，以及一个返回到用户级进程的陷阱。

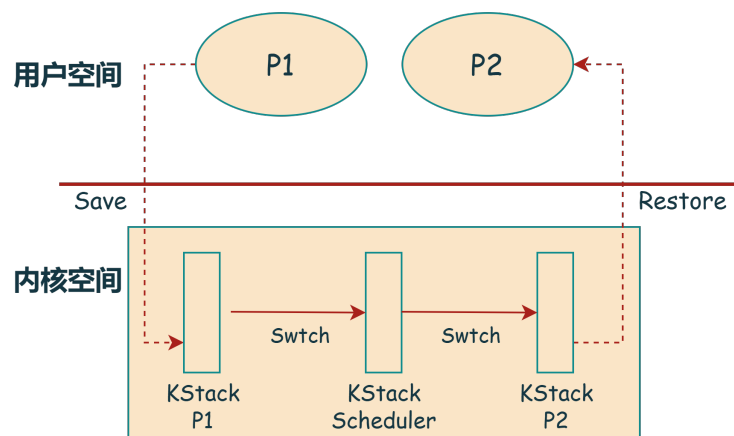


图 2-7 上下文切换

`swtch` 对线程没有直接的了解；它只是保存和恢复上下文（Contexts），`call_sched` 调用 `swtch` 切换到 `cpu->scheduler`，即每个 CPU 的调度程序上下文。调度程序上下文之前通过 `scheduler` 对 `swtch`（`swtch(&p->_context, cpu->get_context());`）的调用进行了保存。当我们追踪 `swtch` 到返回时，它返回到 `scheduler` 而不是 `sched`。

2.4.2.2 有栈协程调度

`Scheduler` 类承担了原本 c 语言书写的 xv6 中的调度任务，存在一种情况使得调度程序对 `swtch` 的调用没有以 `sched` 结束。一个新进程第一次被调度时，它从 `forkret` 开始。`Forkret` 存在以释放 `p->lock`；否则，新进程可以从 `usertrapret` 开始。

Scheduler（运行一个简单的循环：根据优先级找到要运行的进程，运行它直到它让步，然后重复循环。**scheduler** 在进程表上循环查找可运行的进程，比较它们的优先级并选出最高优先级，该进程具有 `p->state == RUNNABLE`。一旦找到一个进程，它将设置 CPU 当前进程变量 `c->proc`，将该进程标记为 `RUNNING`，然后调用 `swtch` 开始运行它。

这种用对象封装两个有栈协程 `call_sched` 和 `start_schedule` 的方法可以使 F7LY 比 xv6 有更清晰和高效的调度方法。

2.4.3 进程内存管理器

在 F7LY 的设计中，考虑到系统调用中涉及大量与进程相关的操作，我们避免将所有下层接口封装到一个单一的管理器类中，以防止类的臃肿，并保持面向对象设计的简洁性与高效性。因此，我们采用了**双管理器**的设计模式来实现进程管理与内存管理的分离。

具体来说，**进程内存管理器**（`ProcessMemoryManager`）是专门用于管理进程的内存操作的类。它封装了许多与进程内存增长、共享内存引用计数等相关的操作，并为内存相关的系统调用提供了清晰的接口。通过这种方式，内存管理的逻辑更加集中，避免了冗余代码，并且提高了代码的可维护性。

在实现上，**进程内存管理器**类不使用全局管理对象，而是为每个进程单独创建一个实例，确保每个进程的内存管理是独立且高效的。这样不仅符合面向对象设计原则，还能确保进程的内存操作彼此隔离，避免了潜在的资源冲突。

```
class ProcessMemoryManager
{
public:
    // 程序段管理
    program_section_desc
    ↪ prog_sections[max_program_section_num];
    int prog_section_count;
    // 堆内存管理
    uint64 heap_start;
    uint64 heap_end;
    // 页表管理
    mem::PageTable pagetable;
    // VMA 管理
    VMA vma_data;
    // 共享标志
```

```

    bool shared_vm;
private:
    // 内存大小
    uint64 total_memory_size;
public:
    //... 各种公共方法
}

```

2.4.3.1 线程的创建与释放

在 F7LY 内核中，进程使用独立的内存管理器来支持线程的创建和释放。虽然线程拥有独立的上下文，但它们位于同一进程中，且共享进程的地址空间。这种设计使得线程间可以高效共享内存，同时确保每个线程的上下文是独立的。为了支持线程的共享内存，我们使用了 `ProcessMemoryManager` 类的多种方法。具体而言，以下方法在 F7LY 中用于支持线程的共享内存操作：

```

ProcessMemoryManager *share_for_thread(); //为线程创建共
→ 享内存（增加引用计数）
ProcessMemoryManager *clone_for_fork(); //为进程创建完
→ 全复制的内存管理器

```

线程的创建与释放流程，尤其在 `fork` 和 `clone` 系统调用中，涉及以下步骤：

1. **分配与初始化子 Pcb**：在调用 `fork` 和 `clone` 时，首先会为子进程分配并初始化一个新的进程控制块（Pcb）。
2. **复制/共享打开文件**：在新创建的 Pcb 中，子进程会继承父进程的文件描述符，这些描述符指向相同的文件对象。
3. **选择内存策略**：根据 `clone` 调用传入的标志位（如 `CLONE_VM`），决定是否共享内存。
 - 不共享内存：如果不共享内存，子进程会创建一个完全独立的内存管理器。此时调用父进程的 `clone_for_fork` 方法，创建一个深度拷贝的内存管理器。在此过程中，系统为子进程分配一个全新的页表，并将 `trampoline`、`sig_trampoline` 以及程序段描述数组

(`prog_sections`) 与计数、堆元数据、VMA 数据等内容进行复制。最终，复制后的进程内存管理器对象会与新进程关联。

- 共享内存: 如果共享内存，子进程会调用父进程的 `share_for_thread` 方法，创建一个指向同一内存管理器的引用，并增加引用计数。这样，子进程和父进程将共享同一块内存区域，从而实现高效的内存使用。

4. **完成初始化与返回**: 在完成进程的内存管理器设置后，继续进行其余的进程初始化工作，最终返回子 Pcb 对象。

2.4.3.2 内存增长与回收

F7LY 通过 `ProcessMemoryManager` 类提供了一系列方法来支持进程内存的动态增长与回收。这些方法涵盖了从堆内存的扩展与收缩，到虚拟内存区域 (VMA) 的管理等多个方面。通过这些接口，F7LY 可以支持 `sbrk`、`brk`、`mmap` 系统调用为进程提供动态内存分配的支持。此外，系统通过 `/proc/stat` 路径暴露进程的内存占用状态，方便外部工具或系统管理员查看进程的内存使用情况。

2.4.3.3 内存释放

在 F7LY 中，进程内存的管理采用了统一的资源释放机制，以确保内存资源能够安全、准确地释放。我们通过 `free_all_memory` 方法来释放进程的所有内存资源，并且在内存资源的释放过程中，确保线程间的共享内存不会发生悬挂引用或重复释放的问题。该方法的工作流程如下：

1. **检查共享标志**: 首先，方法会检查进程内存管理器中的 `shared_vm` 标志，以确定当前进程是否与其他线程共享内存。
2. **处理共享内存**: 在释放内存前，首先会减少共享内存的引用计数。只有当引用计数降为 0 时，内存资源才会被实际释放。这样确保了共享内存仅在最后一个引用被释放时才清理掉，从而避免了早期释放导致的悬挂引用问题。
3. **释放内存资源**: 如果当前进程不共享内存，或者引用计数已经降为 0，方法会继续释放所有与进程相关的内存资源，包括页表、堆内存、VMA 等。通过调用 `pagetable.freewalk_mapped()` 来递归释放页表及其映射的物理页，并清理其他内存区域。

2.4.4 进程管理器

F7LY 的进程管理使用类 `ProcessManager` 封装对进程的许多操作，并使用全局对象 `k_pmm` 进行管理，既包含了对于当前进程的状态信息获取和修改接口，又

作为系统调用和进程结构体这一个个体之间的桥梁便于通过系统调用直接对进程进行操作。

下面将介绍这一类的公共接口与各字段的含义：

```
class ProcessManager
{
private:
    SpinLock _pid_lock;
    SpinLock _wait_lock;
    int _cur_pid;
    Pcb *_init_proc;
    uint _last_alloc_proc_gid;

public:
    ProcessManager() = default;
    // .....
}
```

`ProcessManager` 类的功能可以分为两个主要部分：

1. 进程状态管理：

- 分配与释放进程： `ProcessManager` 负责进程的生命周期管理，包括进程的创建、销毁、以及资源的回收。每当一个新进程需要创建时，它会调用 `ProcessManager` 来分配进程 ID（PID）并初始化进程相关的资源。当进程结束时， `ProcessManager` 会负责清理并回收进程所占用的内存和其他资源。
- 进程属性设置: `ProcessManager` 提供了一系列接口用于设置和获取进程的各种属性，如 PID、PPID、PGID、UID、GID 等。这些属性对于进程的管理和调度至关重要，确保每个进程都能正确地标识和管理其资源。
- 进程状态转换： `ProcessManager` 管理进程的状态转换，包括进程的运行、就绪、阻塞和终止等状态。它通过调度器与进程控制块（PCB）协同工作，确保进程能够在适当的时间点切换状态，并正确响应系统调用和中断。

2. 系统调用接口与进程生命周期：

- 系统调用处理： `ProcessManager` 提供了多个系统调用接口，如 `fork`、

`execve`、`wait`、`exit` 等。这些接口允许用户态程序通过系统调用与内核交互，执行进程创建、程序替换、进程等待和终止等操作。

- 进程调度与切换：`ProcessManager` 与调度器协同工作，管理进程的调度和上下文切换。它确保在多任务环境下，CPU 资源能够公平地分配给各个进程，并根据优先级和状态进行调度。
- 信号处理与进程间通信：`ProcessManager` 还负责处理进程间的信号传递和通信机制，确保进程能够响应外部事件和内部状态变化。

执行用户态程序的流程是：首先使用 `fork` 创建一个子进程，然后通过 `execve` 加载 ELF 文件中的用户程序，将原进程的内存空间和执行上下文替换为新程序的资源，并从 ELF 文件定义的入口地址（`entry`）开始执行。

当用户态程序需要申请内核资源或执行特权操作时，会通过系统调用进入内核。这一过程由硬件触发用户态到内核态的陷入（`usertrap`），在陷入点内核会根据异常码进行判断，并通过 `syscall_handler` 包装逻辑进入具体的系统调用处理流程。

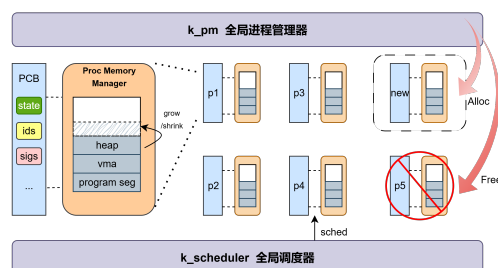


图 2-8 双管理器进程架构

2.5 文件系统架构

2.5.1 虚拟文件系统

虚拟文件系统（Virtual File System, VFS）是操作系统内核中的关键子系统，负责统一管理磁盘文件、I/O 设备等字符流的交互，为上层提供一致、透明的文件访问接口。VFS 通过标准化的系统调用（如 `open()`、`read()`、`write()`）屏蔽了底层文件系统和存储介质的差异，使用户程序能够以统一的方式访问不同类型的文件系统。

在 F7LY 的设计过程中，初赛时参考的往年队伍 XN6 的文件系统存在不完善之处，这使得后续的开发工作遇到了许多困难。尤其是 C++ 的虚类继承机制在文件操作时导致了多次解码，影响了性能。因此，在决赛阶段，F7LY 决定移植成熟

的 `**lwext4 库**`，并基于此实现了一个面向过程调用的 VFS 接口。为了保持高效的性能，我们在虚拟文件管理时使用了一层简单的虚拟文件封装，通过这一层封装来选择正确的文件类型与操作。

2.5.1.1 虚拟文件架构

1. **底层元数据与文件结构**: 在 VFS 的底层, 我们使用了 `lwext4` 库中的 `lwext4_file_struct` 结构体, 它提供了对文件元数据的管理, 包括关键字段:

- `fsize` : 文件大小。
- `fpos` : 当前文件位置读写偏移指针。
- `flags` : 文件标志位, 指示文件的打开模式 (如只读、可写等)。
- `inode` : 指向文件的索引节点 (Inode), 包含文件的元数据。

2. **虚拟文件类 file**: 为了管理打开的文件, F7LY 设计了一个 `file` 类, 该类是一个虚类, 用于表示进程中打开文件的状态属性。 `file` 类包含了以下关键字段:

- `lwext4_file_struct` : 指向底层文件结构的指针, 封装了文件的元数据。
- `st` : 一个 `Kstat` 结构体, 包含文件的状态信息, 如文件大小、访问时间等。
- `attrs` : 一个 `FileAttrs` 结构体, 包含文件的属性信息, 如权限、类型等。
- `file_ptr` : 文件的读写指针, 为支持稀疏文件的高效读写提供了便利。

这个类不仅包含了文件的基本信息, 还通过虚函数为不同的文件类型提供了不同的操作接口。为避免直接操作底层元数据, 这一层的封装同步存储了文件大小等数据, 旨在避免造成 `lwext4` 库的破坏 (该库为 c 语言, 没有封装性), 并在上层能更直观的获取到数据。

3. **虚拟文件类的继承结构**: 根据 Linux “一切皆文件” 的设计理念, F7LY 的 `'file'` 类被派生出了多个具体的文件类型。这些派生类包括:

- `normal_file` : 表示普通文件, 提供了对普通文件的读写操作。
- `device_file` : 表示设备文件, 提供了对设备的特殊操作。
- `pipe_file` : 表示管道文件, 提供了对管道的读写操作。
- `socket_file` : 表示套接字文件, 提供了对网络套接字的操作。
- `dir_file` : 表示目录文件, 提供了对目录的遍历和操作。

- `virtual_file` : 表示虚拟文件，提供对系统状态文件的操作。

每种文件类型根据其特性，提供不同的读写操作。通过虚函数重载，F7LY 能够为每个文件类型实现特定的读写操作，确保文件操作的多样性与高效性。

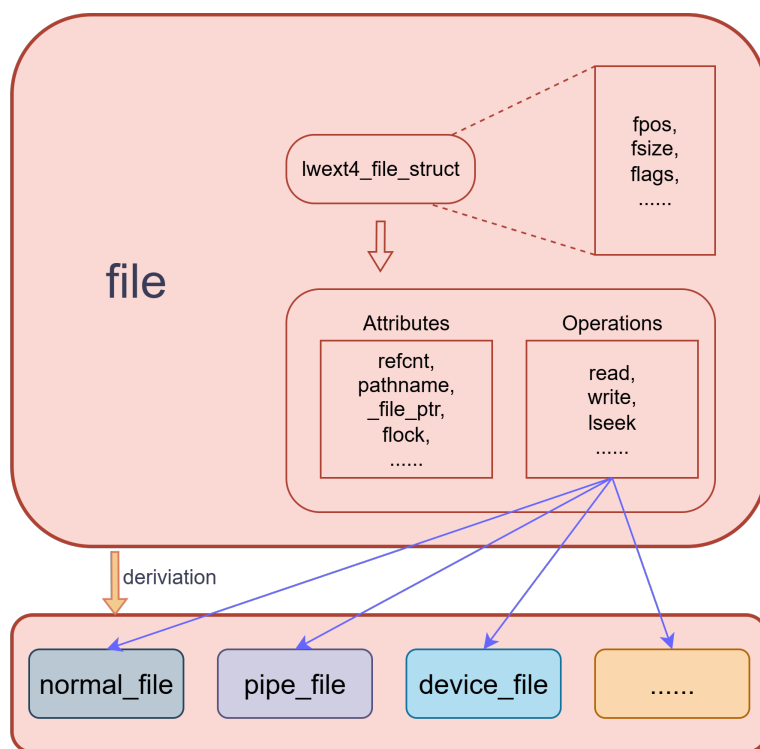


图 2-9 虚拟文件类继承结构

2.5.1.2 虚拟文件系统架构

F7LY 内核目前仅支持 ext4 文件系统，但由于系统状态文件（如 `proc`、`sys` 等）并不存储在 ext4 文件系统中，因此我们设计了一个简单的 VFS 架构来支持多种文件系统的挂载与管理。VFS 通过标准化的接口，屏蔽了底层文件系统的差异，使得用户程序能够以统一的方式访问不同类型的文件系统。在这个 vfs 中，底层的 ext4 文件系统通过 `lwext4` 库进行管理，元数据结构如超级块、inode 等均由该库提供支持，文件的读写操作通过 `normal_file` 类进行封装和管理。而与磁盘的交互则通过 `BlockDevice` 类进行处理，使用 `buf` 结构体统一块大小的访问，确保了对物理存储设备的统一访问接口。而系统状态文件则通过 `virtual_fs` 类进行管理，在初始化时会使用虚拟文件的 `VirtualContentProvider` 来创建和管理这些文件，标记这些文件为动态或静态以便后续与内核状态同步。这样，F7LY 的 VFS 能够同时支持持久化存储的文件系统和非持久化的系统状态文件，提供了灵

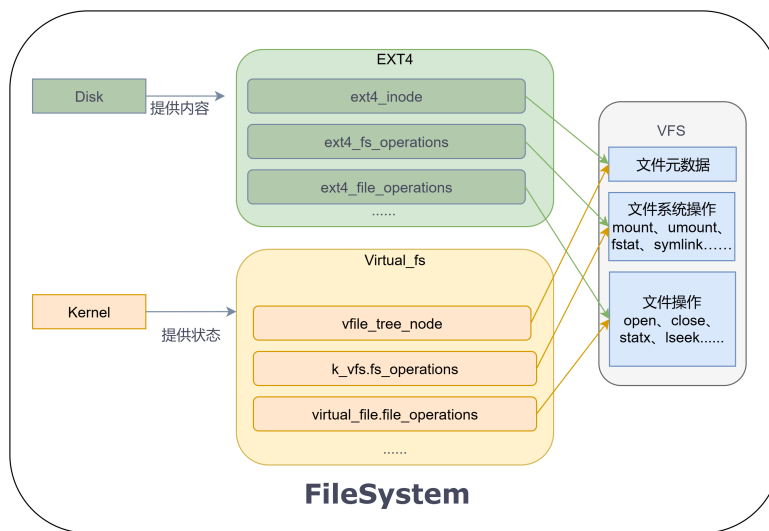


图 2-10 虚拟文件系统架构

活且高效的文件访问机制。目前状态文件支持的路径包括：

- `/proc/`：包含进程信息、系统状态等动态内容。
- `/sys/`：包含系统硬件信息、内核参数等静态内容。
- `/dev/`：包含设备文件，提供对硬件设备的访问接口。

2.5.1.3

2.5.1.4 VFS 与性能的优化

为了确保高效的性能，F7LY 采取了如下优化策略：

- `open` 等全局的文件操作通过使用面向过程调用的 VFS 接口，避免了 C++ 虚类继承机制所带来的性能开销，尤其是在频繁进行文件操作时。
- 虚拟文件管理仅在必要时使用虚拟文件封装，从而确保在文件操作时能够快速定位并执行正确的文件类型操作。
- 文件读写这类基础操作通过虚函数重载的方式来处理不同类型的文件，实现了代码的灵活性和可扩展性，同时确保了操作的高效性。

2.5.2 VFS 核心元数据结构剖析

2.5.2.1 Buffer

Buffer 用于管理磁盘数据的内存缓冲区。它提供了磁盘扇区数据在内存中的抽象表示。F7LY 的缓冲区容器（BufferBlock），组织和管理一组相关的磁盘缓冲区，用于组成了链表节点，串接而成一块缓冲数据链。

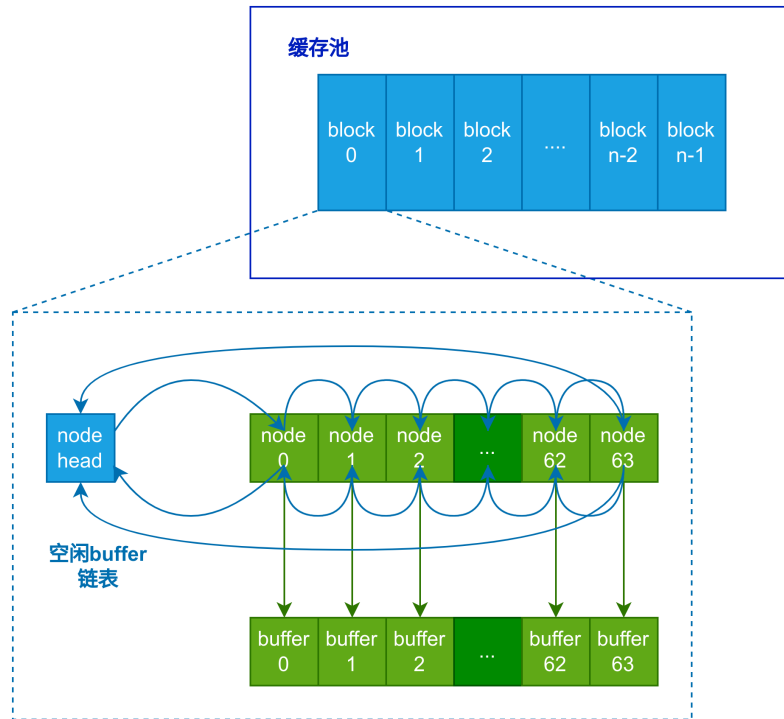


图 2-11 Buffer 缓冲区管理

2.5.2.2 SuperBlock

超级块作为文件系统的核心元数据结构，承担着存储文件系统全局配置信息的重要职责。在物理存储层面，该结构通常映射到磁盘特定位置的元数据存储区域。从面向对象的角度来看，每个超级块实例都代表着一个具体的文件系统实例化对象。

针对基于持久化存储的文件系统，其生命周期管理包含以下关键流程：

- **挂载阶段：**内核需要从磁盘元数据区域读取原始超级块信息，并在内存中构建对应的运行时数据结构；
- **卸载阶段：**系统需要执行相反的操作，包括释放内存中的超级块对象，并将修改后的元数据同步回持久化存储设备。

而对于非持久化文件系统（如内存文件系统 `sysfs`、`procfs` 等），其超级块管理则简化为仅需在内存空间维护独立的元数据结构，无需考虑与物理存储设备的同步问题。这种差异化的实现机制充分体现了 VFS 设计对不同存储介质的良好适应性。

F7LY 的 VFS 超级块由 `superblock` 定义：

```

struct superblock {
    uint8 s_dev; //块设备标识符
    uint32 s_blocksize; //数据块大小，字节单位

    uint32 s_magic; //文件系统的魔数
    uint32 s_maxbytes; //最大文件大小
    struct inode *root; //指根目录

    struct super_operations *s_op;

    SpinLock dirty_lock;
    struct list_head s_dirty_inodes; //脏 inode 表
};

```

对于具体的文件系统，只需要移植自己的超级块结构体，并完善同样的函数实现。如 ext4 的超级块：

```

struct ext4_sbblock {
    uint32_t inodes_count; /* I-nodes count */
    uint32_t blocks_count_lo; /* Blocks count */
    uint32_t reserved_blocks_count_lo; /* Reserved
    ↪ blocks count */
    uint32_t free_blocks_count_lo; /* Free blocks count
    ↪ */
    uint32_t free_inodes_count; /* Free inodes count */
    uint32_t first_data_block; /* First Data Block */
    uint32_t log_block_size; /* Block size */
    uint32_t log_cluster_size; /* Obsoleted fragment
    ↪ size */
    uint32_t blocks_per_group; /* Number of blocks per
    ↪ group */
    uint32_t frags_per_group; /* Obsoleted fragments per
    ↪ group */
    uint32_t inodes_per_group; /* Number of inodes per
    ↪ group */
    uint32_t mount_time; /* Mount time */
    uint32_t write_time; /* Write time */
    uint16_t mount_count; /* Mount count */
};

```

```

uint16_t max_mount_count; /* Maximal mount count */
uint16_t magic; /* Magic signature */
uint16_t state; /* File system state */
uint16_t errors; /* Behavior when detecting errors
↳ */
uint16_t minor_rev_level; /* Minor revision level */
uint32_t last_check_time; /* Time of last check */
... // 省略其他字段
}

```

2.5.2.3 Inode

在 F7LY 中，Inode（索引节点）是文件系统的核心元数据结构，我们通过独立于文件名的 Inode 编号唯一标识每个文件或目录，并在其中存储文件类型、权限、所有者、大小、时间戳、数据块位置等关键信息。F7LY 的底层文件系统采用多级索引机制：直接指针用于快速访问小文件数据，间接指针支持大文件的存储扩展，从而兼顾性能与可扩展性。

在虚拟文件系统层（VFS），F7LY 定义了一个通用的 Inode 结构体，包含文件类型、权限、锁机制、操作函数指针等字段，并通过函数指针实现对不同文件系统的操作抽象。这样，VFS 可以统一管理各种文件系统的 Inode 操作，如读写、锁定、更新等。F7

```

struct inode {
    uint8 i_dev;
    uint16 i_mode; //类型 & 访问权限
    ...//省略其他字段
    SpinLock lock; //测试完成后再换成信号量
    struct inode_operations *i_op; //inode 操作函数
    struct superblock *i_sb;
    struct vfs_ext4_inode_info i_info; //EXT4 inode 结构
};

struct inode_operations {
    void (*unlockput)(struct inode *self);
    void (*unlock)(struct inode *self);
    void (*put)(struct inode *self);
    void (*lock)(struct inode *self);
    void (*update)(struct inode *self);
};

```

```

ssize_t (*read)(struct inode *self, int user_dst,
↳ uint64 dst, uint off, uint n);
int (*write)(struct inode *self, int user_src,
↳ uint64 src, uint off, uint n);
int (*isdir)(struct inode *self); // 是否是
↳ directory
struct inode *(*dup)(struct inode *self);
//For directory
struct inode *(*dirlookup)(struct inode *self, const
↳ char *name, uint *poff);
int (*deletei)(struct inode *self, struct inode
↳ *ip);
int (*dir_empty)(struct inode *self);
struct inode *(*create)(struct inode *self, const
↳ char *name, uchar type, short major, short
↳ minor);
void (*stat)(struct inode *self, struct stat *st);
};

```

2.5.3 系统文件访问

F7LY 内核实现了完整的虚拟文件系统，为应用程序提供了类 Linux 的系统文件接口。虚拟文件系统通过树形结构组织，支持多种文件类型，包括普通文件、符号链接和设备文件。系统在初始化时会创建以下虚拟文件，并标记动态和静态：

/proc 文件系统：

- `/proc/self/exe` - 当前进程可执行文件的符号链接
- `/proc/meminfo` - 系统内存使用信息
- `/proc/cpuinfo` - CPU 硬件信息
- `/proc/version` - 内核版本信息
- `/proc/mounts` - 文件系统挂载信息
- `/proc/self/mounts` - 当前进程的挂载信息
- `/proc/self/cmdline` - 当前进程命令行参数
- `/proc/self/stat` - 当前进程状态统计
- `/proc/self/maps` - 当前进程内存映射信息
- `/proc/self/pagemap` - 当前进程页面映射
- `/proc/self/status` - 当前进程详细状态

- `/proc/stat` - 系统统计信息
- `/proc/uptime` - 系统运行时间
- `/proc/interrupts` - 中断统计信息
- `/proc/sys` 内核参数:
 - - 最大进程 ID
 - `/proc/sys/kernel/shmmax` - 共享内存段最大大小
 - `/proc/sys/kernel/shmmni` - 共享内存段最大数量
 - `/proc/sys/kernel/shmall` - 共享内存总大小限制
 - `/proc/sys/kernel/tainted` - 内核污染状态
 - `/proc/sys/fs/pipe-user-pages-soft` - 管道用户页面软限制
- `/etc` 配置文件:
 - `/etc/passwd` - 用户账户信息
 - `/etc/ld.so.preload` - 动态链接器预加载库配置
 - `/etc/ld.so.cache` - 动态链接器缓存文件
- `/dev` 设备文件:
 - `/dev/null` - 空设备, 丢弃所有写入数据
 - `/dev/zero` - 零设备, 读取时返回零字节
 - `/dev/loop-control` - Loop 设备控制接口
 - `/dev/loop0` - `/dev/loop7` - Loop 块设备, 支持文件系统镜像挂载
 - `/dev/block/8:0` - 块设备文件

特别地, F7LY 的 Loop 设备支持使得系统能够将文件作为块设备进行挂载, 这为文件系统镜像的使用和测试提供了重要支持。通过 Loop 设备, 用户可以挂载 ISO 镜像、磁盘镜像等文件, 极大地扩展了文件系统的灵活性。

这些虚拟文件通过专门的 Provider 类实现, 每个 Provider 负责生成对应文件的内容, 确保了系统信息的实时性和准确性。虚拟文件系统的实现使得 F7LY 能够很好地兼容标准的 Linux 应用程序和系统工具。

2.6 进程间通信

2.6.1 信号机制

信号是操作系统向进程传递异步事件通知的重要手段, 广泛用于异常处理、进程控制和进程间通信。F7LY 借鉴 Linux 的设计, 支持符合 POSIX 标准的信号机制, 并在 PCB 中内置信号处理所需的关键数据结构。

2.6.1.1 核心数据结构

F7LY 定义了以下结构体以支持灵活、兼容的信号处理：

```
namespace signal
{
    struct signal_frame
    {
        sigset_t mask;
        TrapFrame tf;
        signal_frame *next;
    };
    typedef struct sigaction
    {
        void (*sa_handler)(int); // 信号处理函数
        sigset_t sa_mask;        // 处理期间阻塞的信号
        int sa_flags;
    } sigaction;
}
```

这些结构体与 POSIX 标准中的 `siginfo_t` 语义接近，增强了对标准接口的兼容性；

- `signal_frame` 用于保存信号处理过程中的进程上下文；
- `sigaction` 定义每个信号对应的处理行为。

F7LY 的信号机制支持包括 `SIGCHLD` 在内的常用信号。例如，当子进程状态变为 Zombie 时，内核通过发送 `SIGCHLD` 信号告知父进程状态变化，同时提供子进程的 PID、状态码、用户态和内核态运行时间等信息，为父进程调用 `wait4()` 等系统调用获取子进程状态提供必要数据。

2.6.2 信号处理函数

在任务从内核态返回用户态前，需要处理挂起的信号。F7LY 在中断处理器 `usertrap` 中加入 `handle_signal()`，在时钟中断或其他中断后及时执行挂起信号：

```
if (which_dev == 2)
{
    timeslice++;
}
```

```

if (timeslice >= 10)
{
    timeslice = 0;
    printf("yield in usertrap\n");
    proc::k_scheduler.yield();
}
}
handle_signal();
usertrapret();

```

`handle_signal()` 的核心逻辑是：

1. 获取当前进程 PCB;
2. 遍历进程挂起的信号位掩码，检查每个信号的有效性;
3. 根据 `sigaction` 判断信号是否需要自定义处理：
 - 若 `sa_handler == nullptr` ,使用默认处理（例如 SIGKILL 设置 `p->_killed=true`);
 - 否则调用用户自定义的信号处理函数;
4. 处理完当前信号后不会丢失其他信号，通过位掩码保留剩余信号，保证处理安全性。

2.6.2.1 信号上下文保存与恢复

F7LY 实现的信号处理支持完整的上下文保存与恢复：

- 当内核准备进入用户态执行信号处理函数时，会将当前上下文保存到 `sig_trapframe` 中;
- 信号处理完成后，程序需回到原执行状态。此过程使用 `sigreturn` 系统调用恢复上下文。

为了实现从信号处理返回，F7LY 设计了类似 `trampoline` 的机制：

- 信号处理时将 EPC（返回地址）设置到汇编实现的 `sig_trampoline` 函数;
- `sig_trampoline` 唯一作用是触发 `ecall`，调用 `SYS_rt_sigreturn` 系统调用，执行上下文恢复。

这一设计使得 F7LY 在支持自定义信号处理时能保证上下文正确性，安全地完成用户态与内核态的切换。

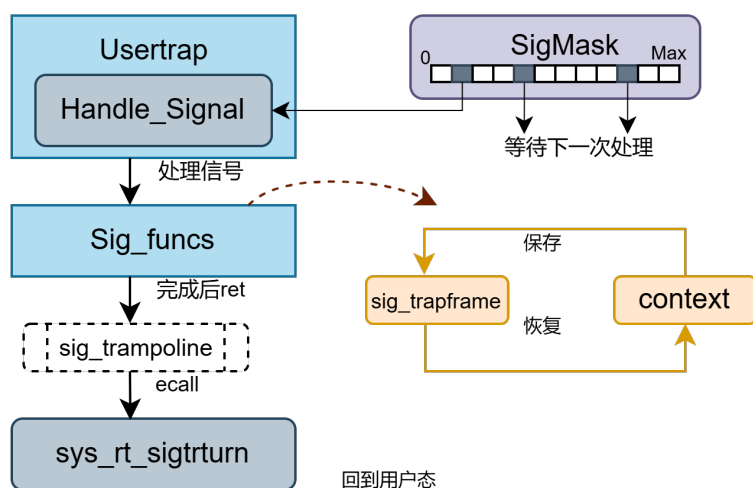


图 2-12 信号处理流程

2.6.3 Futex

Futex (Fast Userspace Mutex) 是 Linux 内核提供了一种高效用户态同步原语，可用于实现互斥锁、条件变量、信号量等多种同步机制。F7LY 借鉴 Linux 的设计，构建了可扩展的 Futex 实现，并支持相关系统调用。

```
struct robust_list {
    struct robust_list *next;
};
struct robust_list_head {
    robust_list list;           // 链表头
    long futex_offset;         // futex 字段的相对偏移
    robust_list *list_op_pending; // 待处理的锁操作
};
```

F7LY 使用 `robust_list` 结构体作为构成单向链表的节点嵌入在用户空间的锁结构中。内核只需要知道前向链接，用户空间可以使用双向链表实现 $O(1)$ 的添加和删除。

- `list` : robust locks 链表的头节点，如果为空则指向自己。
- `futex_offset` : 用户空间设置的相对偏移量，告诉内核 `futex` 字段在数据结构中的位置。
- `list_op_pending` : 防止线程死亡竞争的字段。

- 用户空间首先将此字段设置为即将获取的锁的地址。
- 然后执行锁获取操作。
- 再将自己添加到列表中。
- 最后清除此字段。

Robust Futex 的意义是当线程异常终止时，内核可以自动清理该线程持有的锁，防止死锁。

```
int futex_wait(uint64 uaddr, int val, tmm::timespec
    ↪ *timeout);
int futex_wakeup(uint64 uaddr, int val, void *uaddr2,
    ↪ int val2);
```

Futex 是现代多线程程序同步的基础设施，可用于实现 pthread 库的互斥锁、条件变量、读写锁等高级同步原语，既具备高效的用户态自旋性能，又能在需要阻塞时与内核协作实现调度，让同步操作兼具性能和安全性。

2.6.4 共享内存机制

共享内存是一种高效的进程间通信（IPC）机制，允许多个进程直接访问同一块物理内存区域，从而实现数据的快速交换。F7LY 借鉴 Linux 设计的同时改用面向对象的管理方式，支持符合 POSIX 标准的共享内存机制，并在内核中实现了相关系统调用。F7LY 的内核空间通过类 `SharedMemoryManager`，并设立全局对象 `k_smm` 来管理共享内存区域。该类负责创建、销毁和跟踪共享内存段的生命周期，确保多个进程能够安全地访问和操作共享内存。该类的定义如下：

```
class ShmManager
{
private:
    eastl::unordered_map<int, shm_segment>*
        ↪ segments;
    int next_shmid;
    uint64 shm_base;
    uint64 shm_size;
    eastl::vector<free_block> free_blocks;
public:
    int create_seg(key_t key, size_t size, int
        ↪ shmflg);
```

```

int delete_seg(int shmid);
void *attach_seg(int shmid, void *shmaddr =
    ↪ nullptr, int shmflg = 0);
int detach_seg(void *addr);
bool is_shared_memory_address(void *addr);
bool find_shared_memory_segment(void *addr, void
    ↪ **start_addr, size_t *size = nullptr);
bool add_reference_for_fork(void *addr);
int shmctl(int shmid, int cmd, struct shmid_ds
    ↪ *buf, uint64 buf_addr = 0);
...//省略其他函数
};

```

2.6.4.1 共享内存段结构体

该类封装了一个共享内存段的列表，每个共享内存段由 `shm_segment` 类表示，包含以下关键字段：

```

struct shm_segment
{
    int shmid;           // 共享内存段 ID
    key_t key;           // 共享内存段键值
    size_t size;         // 用户请求的原始大小
    size_t real_size;    // 实际分配的页对齐大小
    union
    {
        u16 shmflg;
        struct
        {
            u16 o_exec : 1; // 其他执行权限
            u16 o_write : 1; // 其他写权限
            u16 o_read : 1; // 其他读权限
            u16 g_exec : 1; // 组执行权限
            u16 g_write : 1; // 组写权限
            u16 g_read : 1; // 组读权限
            u16 u_exec : 1; // 用户执行权限
            u16 u_write : 1; // 用户写权限
            u16 u_read : 1; // 用户读权限
        };
    };
};

```

```

        u16 _rsv : 7;
    }__attribute__((__packed__));
}__attribute__((__packed__));
eastl::vector<pair<tid,void*>> attached_addrs;
uint64 phy_addrs; // 物理地址
int nattch;       // 当前附加的进程数量
....//省略其他字段
};

```

该结构体与 linux 的 `shm_segment` 结构体兼容，包含了共享内存段的 ID、键值、大小、权限标志等信息。与原本的 linux 结构体不同，它维护了一个附加地址列表，用于跟踪所有附加到该共享内存段的虚拟地址并存储附加地址的唯一 tid，该方法可以支持多个进程附加到同一个共享内存段，并使用键值区别不同页表上的附加值。

2.6.4.2 共享内存的实现

在设计 `SharedMemoryManager` 的类方法时，F7LY 直接参考了 posix 标准的共享内存 API，并实现了以下关键方法：

- `create_seg`：该方法对标 `shmget` 系统调用，创建一个新的共享内存段，并返回其 ID。该方法接受三个参数：键值、大小和标志位。它会检查是否已经存在同样的键值的共享内存段，如果存在则返回其 ID，否则创建新的段并返回新段的 ID。
- `delete_seg`：该方法对标 `shmctl` 系统调用中的删除逻辑，但给出更灵活的 api，删除指定的共享内存段。它会检查该段是否存在，并且只有附加地址列表为空，且设置删除标志（`SHM_DEST`）时才会被调用删除，否则减少附加计数并按标准更新段信息。删除后会释放物理内存并从段列表中移除。
- `attach_seg`：该方法对标 `shmat` 系统调用，允许进程附加到指定的共享内存段。它会检查是否已经存在附加地址，如果不存在则分配新的虚拟地址，并将其添加到附加地址列表中。若指定了附加地址，则直接使用该地址。附加的同时记录此次附加的进程 ID，并增加附加计数。若附加成功，则返回附加地址。
- `detach_seg`：该方法对标 `shmdt` 系统调用，允许进程从共享内存段中分离。它会检查指定的地址是否属于共享内存段，并从附加地址列表中移除该地址，同时减少附加计数。如果附加计数为 0 且设置了删除标志，则释放物理内存并从段列表中移除该段。

- `is_shared_memory_address` : 检查指定地址是否属于共享内存段, 返回布尔值。主要用于查找和验证附加地址是否有效, 并进入共享内存处理逻辑。
- `shmctl` : 该方法对标 `shmctl` 系统调用, 提供对共享内存段的控制操作, 如获取段信息、设置权限等。它接受三个参数: 共享内存段 ID、命令和缓冲区地址。根据命令执行不同的操作, 如获取段信息、设置权限等。

这些方法实现了共享内存段的创建、删除、附加和分离等基本操作, 符合 POSIX 标准的共享内存 API 规范。

2.6.4.3 使用 ShmManager 实现 mmap 的共享内存

在 F7LY 的实现中, PCB 没有直接存储共享内存段的引用计数, 而是通过 `SharedMemoryManager` 类来管理所有共享内存段的生命周期。每个进程在附加共享内存段时, 都会调用 `attach_seg` 方法, 该方法会检查是否已经存在附加地址, 并更新附加地址列表。

F7LY 的 `mmap` 系统调用支持共享内存段的映射。通过 `SharedMemoryManager` 类的 `attach_seg` 方法, 进程可以将共享内存段映射到自己的虚拟地址空间中。当进程调用 `mmap` 时, 若包含 flag `MAP_SHARED`, 则会触发共享内存段的附加操作。在一般的 `mmap` 操作时, 会在用户堆顶选取虚拟地址, 并使用 `PhysicalMemoryManager` 分配物理页。若是共享内存段, 则会调用 `create_seg` 方法并传入特定的 key 值创建一个不会重复的共享内存段。并调用 `attach_seg` 方法将共享内存段附加到进程的虚拟地址空间中。

```
bool is_shared_memory_address(void *addr);
int find_shared_memory_segment(void *addr,
void **start_addr, size_t *size = nullptr);
bool add_reference_for_fork(void *addr);
bool duplicate_attachments_for_fork(uint parent_tid,
    → uint child_tid);
```

共享的 `vma` 数据会在 `fork` 时跟随 `ProcessMemoryManager` 一起管理, 进程复制时使用 `is_shared_memory_address` 检查地址是否属于共享内存段, 并通过 `find_shared_memory_segment` 获取共享内存段信息。若是共享内存段, 则调用 `add_reference_for_fork` 增加引用计数, 并在子进程中复制附加地址。此处区分进程与线程的共享内存段管理, 根据传入的 `CLONE_VM` 标志不同进行不同处理, 确保每个进程在 `fork` 时能够正确处理共享内存段的引用计数和附加地址。

2.6.4.4 通过/dev/shm 访问共享内存

F7LY 的共享内存段可以通过 `/dev/shm` 目录访问。在初始化目录时会通过 `VirtualContentProvider` 创建标志为动态的虚拟目录，该目录下的文件对应于共享内存段，每个文件名为共享内存段的键值（key）。用户可以通过标准的文件操作接口（如 `open`、`read`、`write` 等）来访问这些共享内存段。这些访问的请求将会转发到 `ShmManager` 类的相应方法中进行处理。例如，用户可以通过以下方式创建和访问共享内存段：

```
int fd = open("/dev/shm/my_shared_memory",
O_RDWR | O_CREAT, 0666);
if (fd < 0) {
    perror("open");
    return -1;
}
```

特别的，LTP 测试中初始化阶段会使用下面的方法设置好 IPC 的共享内存段，并以此存储测试的结果。

```
static void setup_ipc(void)
{
    size_t size = getpagesize();
    if (access("/dev/shm", F_OK) == 0) {
        snprintf(shm_path, sizeof(shm_path),
            ↪ "/dev/shm/ltp_%s_%d",
                tid, getpid());
    } else {
        .....//其余进程通信逻辑
    }
}
```

这样的方法可以确保每个进程在访问共享内存时都能正确地创建和使用对应的共享内存段。

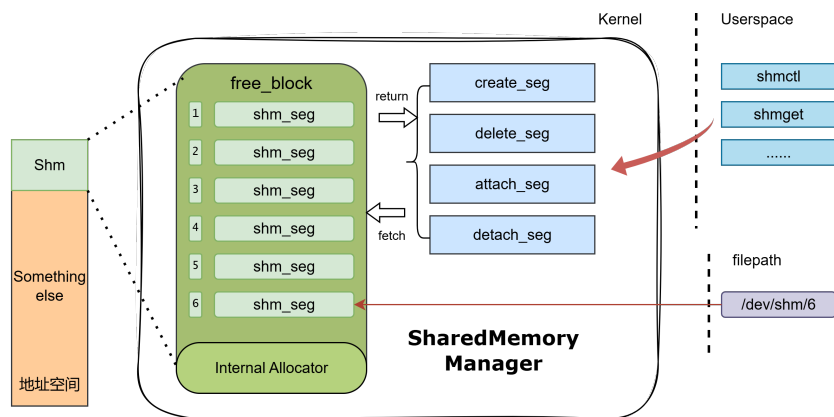


图 2-13 共享内存管理接口

第三章 系统调用

3.1 系统调用概述

系统调用是用户程序与操作系统内核之间的接口，允许用户程序请求内核提供服务，如文件操作、进程管理、内存分配等。F7LY 内核实现了符合 POSIX 标准的系统调用接口，并提供了与 Linux 内核兼容的系统调用列表。当用户态程序需要申请内核资源或执行特权操作时，会通过系统调用进入内核。这一过程由硬件触发用户态到内核态的陷入（`usertrap`），在陷入点内核会根据异常码进行判断，并通过包装逻辑进入具体的系统调用处理流程。在 F7LY 中，系统调用的管理由专用的 `SyscallHandler` 类完成，并在内核启动的 `main` 函数中统一完成系统调用表的初始化绑定。其核心结构示例如下：

```
#define BIND_SYSCALL(sysname)
↪ \
    _syscall_funcs[SYS_##sysname] =
    ↪ &SyscallHandler::sys_##sysname; \
    _syscall_name[SYS_##sysname] = #sysname;

class SyscallHandler
{
    using SyscallFunc = uint64
    ↪ (SyscallHandler::*)(void);
private:
    SyscallFunc
    ↪ _syscall_funcs[max_syscall_funcs_num];
    const char
    ↪ *_syscall_name[max_syscall_funcs_num];
    uint64_t _default_syscall_impl();
    .....
public:
    void init(); // 使用构造函数进行
    ↪ init
    void invoke_syscaller(); // 调用系统调用
private:
    int _fetch_addr(uint64 addr, uint64 &out_data);
    int _fetch_str(uint64 addr, eastl::string &buf,
    ↪ uint64 max);
```

```

uint64 _arg_raw(int arg_n);
int _arg_int(int arg_n, int &out_int);
int _arg_long(int arg_n, long &out_int);
int _arg_addr(int arg_n, uint64 &out_addr);
int _arg_str(int arg_n, eastl::string &buf, int
    ↪ max);
int _arg_fd(int arg_n, int *out_fd, fs::file
    ↪ **out_f);
}
extern SyscallHandler k_syscall_handler;

```

3.2 系统调用流程

3.2.1 系统调用执行流程

F7LY 内核的系统调用执行流程可以分为以下几个关键步骤：

1. **用户态触发**: 用户程序通过 `ecall` 指令 (RISC-V) 或 `syscall` 指令 (LoongArch) 触发系统调用, 产生异常并陷入内核态。
2. **异常处理**: 硬件自动保存用户态上下文, 并跳转到内核的异常处理入口 `usertrap`。
3. **系统调用分发**: 内核根据异常码判断为系统调用异常, 调用 `syscall_handler` 进行系统调用分发。
4. **参数获取**: 通过寄存器获取系统调用号和参数, 调用 `SyscallHandler::invoke_syscaller` 方法。
5. **函数查表执行**: 根据系统调用号在 `_syscall_funcs` 数组中查找对应的处理函数并执行。
6. **返回用户态**: 将执行结果放入返回寄存器, 恢复用户态上下文并返回。

3.2.2 参数获取机制

F7LY 内核提供了一套完整的参数获取机制, 通过 `SyscallHandler` 类的私有方法实现:

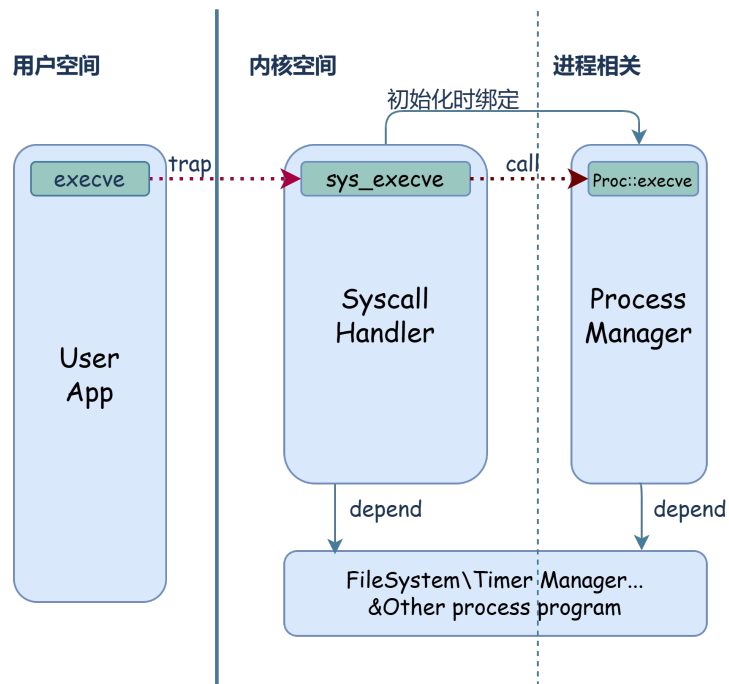


图 3-1 系统调用示意图

```

// 获取原始参数（从寄存器直接读取）
uint64 SyscallHandler::_arg_raw(int arg_n) {
    // 从当前进程的 trapframe 中获取第 arg_n 个参数
    // RISC-V: a0-a5 寄存器存储前 6 个参数
    // LoongArch: $a0-$a5 寄存器存储前 6 个参数
    return current_proc()->trapframe->regs[10 + arg_n];
    ↪ // a0 = x10
}

// 获取整型参数
int SyscallHandler::_arg_int(int arg_n, int &out_int) {
    out_int = (int)_arg_raw(arg_n);
    return 0;
}

// 获取地址参数并验证
int SyscallHandler::_arg_addr(int arg_n, uint64
    ↪ &out_addr) {
    out_addr = _arg_raw(arg_n);
    // 验证地址是否在用户空间范围内

```

```

    if (out_addr >= MAXVA) {
        return -1; // 地址无效
    }
    return 0;
}

// 获取字符串参数
int SyscallHandler::_arg_str(int arg_n, eastl::string
↪ &buf, int max) {
    uint64 addr;
    if (_arg_addr(arg_n, addr) < 0) {
        return -1;
    }
    return _fetch_str(addr, buf, max);
}

// 从用户空间获取字符串
int SyscallHandler::_fetch_str(uint64 addr,
↪ eastl::string &buf, uint64 max) {
    buf.clear();
    for (uint64 i = 0; i < max; i++) {
        char c;
        if (copyinstr(current_proc()->pagetable,
↪ (char*)&c, addr + i, 1) < 0) {
            return -1; // 页面错误
        }
        if (c == '\\0') {
            break;
        }
        buf.push_back(c);
    }
    return 0;
}

```

3.2.3 系统调用表初始化

系统调用表的初始化在内核启动时的 `main` 函数中完成,通过 `SyscallHandler::init()` 方法实现:

```

void SyscallHandler::init() {
    // 初始化所有系统调用为默认实现
    for (int i = 0; i < max_syscall_funcs_num; i++) {
        _syscall_funcs[i] =
            ↪ &SyscallHandler::_default_syscall_impl;
        _syscall_name[i] = "unknown";
    }

    // 使用宏绑定具体的系统调用实现
    BIND_SYSCALL(read);           // sys_read
    BIND_SYSCALL(write);          // sys_write
    BIND_SYSCALL(open);           // sys_open
    // ... 更多系统调用绑定
}

// 默认系统调用实现
uint64_t SyscallHandler::_default_syscall_impl() {
    uint64 syscall_num = _arg_raw(-1);
    kprintf("[SYSCALL] Unimplemented syscall: %d\n",
        ↪ syscall_num);
    return -ENOSYS; // 返回" 功能未实现" 错误码
}

```

3.2.4 系统调用分发器

系统调用的核心分发逻辑由 方法实现：

```

void SyscallHandler::invoke_syscaller() {
    uint64 syscall_num = _arg_raw(-1);
    // 边界检查
    if (syscall_num >= max_syscall_funcs_num) {
        current_proc()->trapframe->regs[10] =
            ↪ _default_syscall_impl();
        return;
    }

    // 获取对应的系统调用函数指针
    SyscallFunc func = _syscall_funcs[syscall_num];
}

```

```

// 执行系统调用并将结果存储到 a0 寄存器
uint64 result = (this->*func)();
current_proc()->trapframe->regs[10] = result;
// 记录系统调用执行信息（调试模式）
#ifdef DEBUG_SYSCALL
kprintf("[SYSCALL] %s(%d) -> %ld\n",
        _syscall_name[syscall_num], syscall_num,
        result);
#endif
}

```

这种设计的优势在于：

- **统一性**：所有系统调用都通过统一的接口进行分发
- **可扩展性**：通过简单的宏定义即可添加新的系统调用
- **容错性**：未实现的系统调用会调用默认实现，避免内核崩溃
- **调试友好**：提供系统调用名称和编号的映射，便于调试

3.3 系统调用实现

目前 F7LY 内核已经实现并验证了超过 210 个系统调用，涵盖了进程管理、内存管理、文件操作、网络通信、信号处理等核心功能。每个系统调用都通过 `SyscallHandler` 类的成员函数实现，并在内核启动时进行绑定。本节按照功能模块分类，按常用程度介绍主要系统调用的实现。

3.3.1 进程生命周期管理

进程生命周期管理是操作系统的核心功能之一，F7LY 实现了完整的进程创建、执行、等待和终止机制。

3.3.1.1 进程创建与执行

- `sys_fork()`：创建新进程，完全复制父进程的地址空间、文件描述符表和进程上下文。新进程获得新的 PID，与父进程并发执行。
- `sys_clone()`：Linux 风格的进程/线程创建接口，支持更细粒度的资源共享控制。可以指定哪些资源在父子进程间共享（如内存空间、文件描述符、信号处理等）。
- `sys_clone3()`：clone 的扩展版本，提供更多的创建选项和参数。

- `sys_execve()` : 加载并执行新程序, 替换当前进程映像。支持参数传递 (`argv`) 和环境变量设置 (`envp`), 通过 ELF 加载器实现程序加载。
- `sys_exec()` : 简化版的程序执行接口。

3.3.1.2 进程等待与终止

- `sys_wait4()` : 等待子进程状态改变, 获取退出状态。支持阻塞等待和非阻塞查询, 可指定等待特定 PID 的子进程。
- `sys_wait()` : 简化版的等待接口, 等待任意子进程终止。
- `sys_waitid()` : 更灵活的等待接口, 支持等待多种进程状态改变事件。
- `sys_exit()` : 正常终止当前进程, 设置退出码并释放进程资源。
- `sys_exit_group()` : 终止整个线程组 (进程组), 用于多线程程序的统一退出。

3.3.1.3 进程标识与信息获取

- `sys_getpid()` : 获取当前进程 ID。
- `sys_getppid()` : 获取父进程 ID。
- `sys_gettid()` : 获取当前线程 ID。
- `sys_getuid()` / : 获取用户 ID 和有效用户 ID。
- `sys_getgid()` / : 获取组 ID 和有效组 ID。
- `sys_getpgid()` / `sys_setpgid()` : 进程组管理。
- `sys_setsid()` : 创建新会话并设置进程组长。

3.3.2 内存管理

内存管理系统调用提供了虚拟内存映射、动态内存分配和内存保护等功能。

3.3.2.1 内存映射与解映射

- `sys_mmap()` : 内存映射核心接口, 支持文件映射、匿名映射、共享内存映射。可指定映射地址、保护属性 (读/写/执行) 和映射标志。
- `sys_munmap()` : 解除内存映射, 释放指定地址范围的虚拟内存。
- `sys_mremap()` : 重新映射内存区域, 支持扩展或收缩映射大小。
- `sys_mprotect()` : 修改内存页的保护属性, 实现内存访问控制。
- `sys_madvise()` : 向内核提供内存使用建议, 优化内存管理策略。
- `sys_msync()` : 同步内存映射文件的修改到存储设备。

3.3.2.2 堆内存管理

- `sys_brk()`：调整数据段大小，用于堆内存的扩展和收缩。是 `malloc` 库的底层实现基础。

3.3.2.3 共享内存 (System V IPC)

- `sys_shmget()`：创建或获取共享内存段。
- `sys_shmat()`：将共享内存段连接到进程地址空间。
- `sys_shmdt()`：从进程地址空间分离共享内存段。
- `sys_shmctl()`：共享内存控制操作（获取状态、删除等）。

3.3.3 文件系统操作

文件系统操作是系统调用中最丰富的类别，提供了完整的文件和目录操作接口。

3.3.3.1 文件打开与关闭

- `sys_openat()`：相对目录打开文件，支持各种打开标志（只读、只写、读写、创建、追加等）。
- `sys_openat2()`：扩展版本，提供更多的打开选项和安全控制。
- `sys_close()`：关闭文件描述符，释放相关资源。
- `sys_close_range()`：批量关闭一个范围内的文件描述符。

3.3.3.2 文件读写操作

- `sys_read()`：从文件描述符读取数据到缓冲区。
- `sys_write()`：将缓冲区数据写入文件描述符。
- `sys_readv()` / `sys_writev()`：矢量读写，支持多个缓冲区的批量读写。
- `sys_pread64()` / `sys_pwrite64()`：定位读写，在指定偏移位置进行读写操作，不改变文件位置指针。
- `sys_preadv()` / `sys_pwritev()`：定位矢量读写。
- `sys_lseek()`：设置文件位置指针。
- `sys_sendfile()`：在两个文件描述符间高效传输数据，零拷贝实现。

3.3.3.3 文件状态与属性

- `sys_fstat()`：获取文件描述符对应文件的状态信息。
- `sys_fstatat()`：获取相对路径文件的状态信息。

- `sys_statx()` : 扩展的文件状态查询接口, 提供更丰富的文件信息。
- `sys_faccessat()` / `sys_faccessat2()` : 检查文件访问权限。
- `sys_readlinkat()` : 读取符号链接的目标路径。

3.3.3.4 目录操作

- `sys_mkdirat()` : 创建目录。
- `sys_getdents64()` : 读取目录项, 用于 ls 命令的实现。
- `sys_getcwd()` : 获取当前工作目录。
- `sys_chdir()` : 改变当前工作目录。
- `sys_fchdir()` : 通过文件描述符改变工作目录。

3.3.3.5 文件系统管理

- `sys_mount()` : 挂载文件系统。
- `sys_umount2()` : 卸载文件系统。
- `sys_statfs()` / `sys_fstatfs()` : 获取文件系统统计信息。
- `sys_sync()` / `sys_fsync()` / `sys_fdatasync()` : 文件系统同步操作。

3.3.4 信号处理

F7LY 实现了完整的 POSIX 信号机制, 支持信号的发送、捕获和处理。

3.3.4.1 信号发送

- `sys_kill()` : 向进程发送信号。
- `sys_tkill()` : 向特定线程发送信号。
- `sys_tgkill()` : 向特定线程组中的线程发送信号。
- `sys_rt_sigqueueinfo()` : 发送带数据的实时信号。

3.3.4.2 信号处理设置

- `sys_rt_sigaction()` : 设置信号处理函数。
- `sys_rt_sigprocmask()` : 设置信号屏蔽字。
- `sys_rt_sigpending()` : 查询待处理信号。
- `sys_rt_sigsuspend()` : 临时设置信号屏蔽字并等待信号。
- `sys_rt_sigtimedwait()` : 等待指定信号, 支持超时。
- `sys_rt_sigreturn()` : 从信号处理函数返回。
- `sys_sigaltstack()` : 设置备用信号栈。

3.3.5 网络通信 (Socket)

F7LY 提供了完整的 BSD Socket API 实现，支持 TCP/UDP 网络通信。

3.3.5.1 Socket 创建与管理

- `sys_socket()` : 创建 socket。
- `sys_socketpair()` : 创建 socket 对，用于进程间通信。

3.3.5.2 网络连接

- `sys_bind()` : 绑定 socket 到本地地址。
- `sys_listen()` : 监听连接请求。
- `sys_accept()` / `sys_accept4()` : 接受连接请求。
- `sys_connect()` : 主动建立连接。

3.3.5.3 数据传输

- `sys_sendto()` / `sys_recvfrom()` : UDP 数据传输。
- `sys_sendmsg()` / `sys_recvmsg()` : 复杂消息传输。
- `sys_getsockname()` / `sys_getpeername()` : 获取 socket 地址信息。
- `sys_setsockopt()` / `sys_getsockopt()` : socket 选项设置和查询。

3.3.6 时间管理

3.3.6.1 时间获取与设置

- `sys_gettimeofday()` : 获取当前时间。
- `sys_clock_gettime()` / `sys_clock_settime()` : 高精度时间操作。
- `sys_clock_getres()` : 获取时钟精度。
- `sys_times()` : 获取进程时间统计。
- `sys_uptime()` : 获取系统运行时间。

3.3.6.2 定时器与睡眠

- `sys_nanosleep()` / `sys_clock_nanosleep()` : 高精度睡眠。
- `sys_timer_create()` / `sys_timer_settime()` / `sys_timer_delete()` : POSIX 定时器。
- `sys_setitimer()` : 间隔定时器。

3.3.7 系统信息与控制

- `sys_uname()` : 获取系统信息。
- `sys_sysinfo()` : 获取系统运行状态。
- `sys_getrusage()` : 获取资源使用统计。
- `sys_prctl()` : 进程控制操作。
- `sys_sched_yield()` : 主动让出 CPU。
- `sys_sched_setscheduler()` / `sys_sched_getscheduler()` : 调度策略设置。
- `sys_sched_setaffinity()` / `sys_sched_getaffinity()` : CPU 亲和性设置。

3.3.8 其他重要系统调用

3.3.8.1 管道与重定向

- `sys_pipe2()` : 创建管道。
- `sys_dup()` / `sys_dup3()` : 复制文件描述符。

3.3.8.2 同步原语

- `sys_futex()` : 快速用户空间互斥锁。

第四章 总结与展望

4.1 工作总结

1. 完成 RISC-V 与 LoongArch 双架构支持，包括：
 - 串口驱动；
 - PCI 设备管理；
 - 中断控制器配置；
 - 硬盘读写；
 - 并实现与硬件逻辑相匹配的中断处理流程。
 - 完成网络协议栈的驱动适配，支持 TCP/IP 协议栈的基本功能。
2. 通过面向对象设计，完成进程管理和内存管理模块，实现底层架构无关性，达到松耦合结构，为双架构扩展奠定基础，并支持线程任务的创建与调度。
3. 实现独立的虚拟文件系统层，成功将文件系统逻辑与内核解耦，并为支持更多文件系统格式提供继承接口。
4. 完成 Busybox 和 libc 的适配，补充和完善系统调用接口与相关功能。
5. 通过初步性能测试，验证系统具备良好的调度和 I/O 性能表现。
6. 多方法实现进程间通信（IPC）机制和信号处理功能，支持简单的进程协作。
7. 部分通过 Linux Test Project（LTP）压力测试，验证内核稳定性和兼容性。
8. 构建多等级调试日志系统，并支持内核崩溃时打印调用栈回溯（backtrace）及参数信息，方便问题定位。

4.2 经验总结

1. 积累从零开始构建内核的系统化经验，包括：
 - 完成机器引导；
 - 初始化串口输出；
 - 搭建内存布局；
 - 编写中断处理；
 - 实现最小可用内核；

- 逐步完善核心功能。
2. 系统调用的实现结合《System Calls Manual》和参考其他优秀项目成果，符合 Linux 原生 raw syscall 规范。
 3. 不断积累调试经验，包括：
 - 用户态程序的单步调试；
 - 利用 GDB 跟踪调用栈；
 - 精准定位出错变量；
 - 依靠完善的日志获取运行期关键信息。

4.3 未来计划

1. 优化内存布局，进一步提升内存空间利用率。
2. 重构代码结构，提升代码可读性和可维护性。
3. 完善内核模块化设计，支持更多文件系统格式和设备驱动。
4. 增强进程间通信机制，支持更复杂的 IPC 模型，如消息队列、信号量等。
5. 完善多线程支持，优化线程调度算法，提升多核处理器性能。
6. 增强网络协议栈，支持更多协议和功能，如 IPv6、TLS 等。
7. 完善系统调用接口，确保与 Linux 兼容性，支持更多 POSIX 标准。
8. 完善硬件支持，争取能够在 LoongArch 和 RISC-V 物理硬件上适配运行。
9. 针对 Linux Test Project (LTP) 压力测试结果，改进内核结构以提升稳定性和兼容性。

附录 A 一些重要的类的具体字段解析

表 a-1 ProcessManager 接口函数说明（第一部分）

函数名	含义
get_cur_pcb	获取当前执行中的进程控制块
alloc_pid	为进程分配进程 ID
alloc_proc	从进程池中寻找一个空进程并分配
set_killed	将 killed 字段设为 1，等待杀死进程
proc_pagetable	为进程分配一个页表的空间并进行映射
proc_freepagetable	释放一个进程的页表并回收空间
either_copy_in	从用户空间或内核空间拷贝数据到内核空间
either_copy_out	从内核空间或用户空间拷贝数据到用户空间
进程操作	
execve	执行新程序
growproc	扩展进程内存，由 brk 等函数调用
wait4	等待子进程结束
load_seg	加载程序段到页表映射的虚拟内存中，在执行程序时使用
sleep	使进程进入睡眠状态，等待某个通道或锁
wakeup	唤醒在指定通道或锁上睡眠的进程

表 a-2 ProcessManager 接口函数说明（第二部分）

exit_proc	真正退出进程的逻辑，设置状态并清理资源
exit	当前进程或线程退出（只退出自己）
exit_group	当前线程组（或进程组）全部退出
clone	创建新进程或线程，支持克隆和线程局部存储
fork	创建新进程，返回新进程的 PID
brk	扩展进程地址，设置进程的堆顶地址
文件操作	
open	打开文件或设备，返回文件描述符
mkdir	在指定路径下创建目录
close	关闭文件描述符
fstat	获取指定文件描述符下的文件状态信息
chdir	改变当前工作目录
getcwd	获取当前工作目录路径
mmap	映射文件到内存
munmap	解除内存映射
unlink	删除文件或目录的链接
pipe	创建管道，返回读写文件描述符
其他系统调用	
reparent	托孤机制，重新设置进程的父进程为 init
prlimit64	设置或获取进程资源限制
set_robust_list	设置鲁棒列表，用于处理线程安全的锁
set_tid_address	设置线程 ID 地址，用于线程同步和管理

表 a-3 PCB 结构字段说明（第一部分）

字段名	类型	含义
<code>_lock</code>	SpinLock	进程控制块的锁，用于并发访问控制
<code>_gid</code>	int	全局 ID，用于在进程池中唯一标识进程
文件系统相关		
<code>_cwd</code>	dentry*	当前工作目录
<code>_cwd_name</code>	string	工作目录名称
<code>_ofile</code>	file*	进程打开的文件列表 (文件描述符 文件结构)
<code>_fl_cloexec</code>	bool	记录每个文件描述符的 close-on-exec 标志
<code>exe</code>	string	可执行文件的绝对路径
进程状态信息		
<code>_state</code>	enum Procstate	进程当前状态
<code>_chan</code>	void*	进程睡眠时等待的通道 (例如：某个锁或事件)
<code>_killed</code>	int	进程是否被标记为 kill (非零表示被 kill)
<code>_xstate</code>	int	进程退出状态，用于父进程 wait() 获取
<code>_pid</code>	int	进程 ID (Process ID)
<code>_parent</code>	Pcb*	父进程的 PCB 指针
<code>_name</code>	char	进程名称 (用于调试)

表 a-4 PCB 结构字段说明（第二部分）

字段名	类型	含义
内存管理相关		
<code>_kstack</code>	uint64	内核栈的虚拟地址
<code>_sz</code>	uint64	进程用户空间的内存大小 (字节)
<code>_pt</code>	PageTable	用户空间的页表
<code>_trapframe</code>	TrapFrame	保存用户态 TrapFrame 的地址 (用于系统调用和异常处理)
<code>_context</code>	Context	保存进程的上下文信息 (寄存器等), 用于进程切换
调度相关		
<code>_slot</code>	int	分配给进程的时间片剩余量
<code>_priority</code>	int	进程优先级 (0 最高, 19 最低)
消息队列		
<code>_mqmask</code>	uint	用于标记进程使用的消息队列
虚拟内存区域 (VMA)		
<code>_vm[NVMA]</code>	vma	虚拟内存区域数组
线程/futex 相关		
<code>_futex_addr</code>	void *	用于 futex 的唤醒
<code>_set_child_tid</code>	int*	线程的标志 id
<code>_clear_child_tid</code>	int *	清除线程表示 id 的 flag
<code>_robust_list</code>	robust_list_head *	用于处理线程的 robust futexes
信号处理相关		
<code>_sigactions[]</code>	sigaction *	信号处理函数数组
<code>sigmask</code>	uint64	信号掩码
<code>_signal</code>	int	进程等待的信号